

POST-INCIDENT REVIEW

PIR-2026-0621 CONFIDENTIAL -- INTERNAL

The Eleven Dark Hours

A Forensic Analysis of the RMHStudios.com Outage

Sunday, 21 June 2026 00:00 – 11:00 UTC

Subject. A complete root-cause reconstruction of an eleven-hour production outage that interrupted a four-nines availability record, derived directly from four production deployment logs (0a659e7b, 0a6e5ddd, 2e46b0cd, 02493f03). The report covers the technical fault chain, the supporting mathematics of availability and reliability, the financial and client-retention consequences in the immediate wake of the **RMH Capital** launch and the **RMH Space** teaser, and a remediation programme. It opens with a personal letter from the Chief Financial Officer.

Document ID	PIR-2026-0621-v1.0
Classification	Confidential — Internal Only
Prepared by	Reliability Engineering, RMHStudios
Sponsoring officer	Alex Wu, Chief Financial Officer
Incident window	2026-06-21 00:00 – 11:00 UTC (11h 00m)
Primary evidence	4 production deploy logs (8,249 lines)
Status	Closed — corrective actions tracked

A Letter from the Chief Financial Officer

Office of the Chief Financial Officer

RMHStudios | 21 June 2026

To our clients, our partners, our investors, and the team who builds RMHStudios.com every day,

On Sunday, 21 June 2026, between midnight and 11:00 in the morning, RMHStudios.com was unavailable. For eleven hours the front door to everything we have asked you to believe in was locked, and the people who tried to walk through it were turned away with nothing but a timeout. I am writing to you directly, in my own name, because an outage of this length is not a footnote in an engineering ticket. It is a broken promise, and the promise was mine to keep as much as anyone's.

I will not hide behind the passive voice. We changed how we deploy the website. A new “blue/green” release mechanism was rolled out in the early hours of Sunday morning. On its very first run it tore down the running web server before its replacement was able to stand up, and the replacement never became healthy because it could not reach the database in time. By the time a human could intervene and put the old server back, the morning was nearly gone. The detailed account of how that happened — down to the exact second, the exact commit, and the exact line of the script — fills the chapters that follow. I have asked the team to hide nothing.

The timing compounds the injury, and I feel that acutely. We had just launched **RMH Capital** and teased **RMH Space**. We asked you to trust us with more — with your capital, with your imagination of where we are going next — and in the same breath we could not keep a single web server answering the phone. Some of you, including clients whose names I know personally, drew the obvious conclusion and stepped back. I do not blame you. Trust is a balance sheet like any other, and on Sunday we ran a deficit.

We held a 99.99% availability standard — the “four nines” that allow roughly fifty-three minutes of downtime in an *entire year*. In one morning we spent more than twelve years of that budget. The arithmetic is unforgiving and it is laid out plainly in Chapter 8. I will not pretend the number is anything other than what it is.

Here is what I can promise in return. First, accountability sits with leadership, not with whoever was on call. Second, every corrective action in Chapter 16 has a named owner and a date, and I will personally review them. Third, we are crediting affected enterprise accounts under our service-level commitments without requiring you to ask. And fourth, this document exists at all because we would rather show you the wound in full than describe it vaguely — that is the only version of transparency I find worth the word.

To the clients who left: the door is open, and it answers now. To those who stayed: thank you for the patience we did not earn that morning. To our team: you fixed this, and the failure was in the system we gave you, not in you.

With sincere apology and full accountability,

Alex Wu

Alex Wu

Chief Financial Officer, RMHStudios

This letter is a personal statement of accountability from the Office of the CFO and forms part of the official incident record PIR-2026-0621.

Executive Summary

RMHStudios.com suffered a complete, customer-facing outage lasting approximately eleven hours, from **00:00 to 11:00 UTC on Sunday, 21 June 2026**. The site returned empty replies and connection timeouts for the entire window. This document reconstructs the event from primary evidence: four production deployment logs captured during and after the incident.

What happened, in one paragraph

A new **blue/green** deployment strategy for the web container was merged to `main` and auto-deployed at **05:13 UTC** (commit `0a659e7b`). The deploy script updated itself mid-run and, on this first execution of the new path, the container-orchestration step *still tore down the live web container* (“blue”) while simultaneously recreating the entire background worker fleet. The replacement container (“green”) was then started, but it never passed its health check: it could not obtain database connections and emitted a sustained flood of `prisma:error Connection terminated due to connection timeout`. With blue destroyed and green unhealthy, **nothing was serving traffic**. The automated rollback logic correctly declined to move traffic — but there was no healthy “blue” left to fall back to. Service was restored by manual intervention around 11:00. Three later deploys that day (11:41, 14:45, 15:02) confirmed a *second, independent* latent fault: even when green became healthy, the cutover could not complete because the Apache active-configuration file sits on a read-only mount and the deploy user is denied both direct writes and `sudo` (the sandbox sets the `NoNewPrivileges` flag).

Key Finding

The outage was caused by the **conjunction of two independent faults**, either of which alone would have been survivable: (A) a release-ordering race that destroyed the live container before a healthy replacement existed, amplified by a database connection storm; and (B) a privilege/mount misconfiguration that prevents the traffic cutover from ever completing. A blue/green system is only as safe as its *weakest* guarantee, and on 21 June both guarantees failed at once.

Impact at a glance

The five numbers that matter

The remainder of this document expands each of these claims with evidence, mathematics, and figures, and closes with a remediation programme (Chapter 14) whose every line item carries an owner and a due date.

Table 1: Incident PIR-2026-0621 — summary of impact.

Service	RMHStudios.com (primary public web property)
Severity	SEV-1 (full outage of a customer-facing system)
Window	2026-06-21 00:00–11:00 UTC ($\approx$ 11h 00m / 660 minutes)
Trigger	Auto-deploy of commit 0a659e7b (blue/green rollout)
Root cause A	Live container removed before healthy replacement; DB connection storm
Root cause B	Apache cutover blocked: read-only conf + sudo denied (NoNewPrivileges)
Detection	Synthetic/manual; no automatic rollback to a healthy blue
Recovery	Manual restoration of the legacy single-container web path
Availability hit	99.99% annual budget (52.6 min) exceeded $\approx 12.5\times$ in one event
Business context	Immediately followed RMH Capital launch and RMH Space teaser
Client impact	Churn of several key accounts; reputational damage at a sensitive moment
Data loss	None; database integrity preserved (migrations were a no-op)

Table 2: Headline metrics referenced throughout this report.

#	Quantity	Value
1	Outage duration	660 min
2	Annual downtime permitted at 99.99%	52.6 min
3	Error-budget overspend (this event alone)	12.55 $\times$
4	Resulting availability over trailing 12 months	$\approx$ 99.87%
5	Health-check failure flood (green, run 0a659e7b)	60s of timeouts

Contents

A Letter from the Chief Financial Officer	1
Executive Summary	3
1 Introduction and Scope	13
1.1 Purpose of this document	13
1.2 Primary evidence	13
1.3 Scope and non-scope	14
1.4 A note on the DeepSeek 401 errors	14
1.5 How to read this report	14
2 System Architecture	15
2.1 The request path	15
2.2 The container fleet	15
2.3 Blue/green deployment, as intended	16
3 Incident Timeline	17
3.1 Narrative timeline	17
3.2 Phases of the incident	18
4 Root Cause A: A Live Container Removed Before a Healthy Replacement Existed	20
4.1 The evidence, stated plainly	20
4.2 Why the health check never passed	20
4.3 The connection storm	21
4.4 The 5-Whys	21
5 Root Cause B: The Cutover That Could Never Complete	23
5.1 The evidence across three deploys	23
5.2 Anatomy of the permission wall	23
5.3 Why this did not cause a second outage	24
5.4 The correct fixes	24
6 The Trigger: A Self-Updating Deploy Script	26
6.1 The line that explains everything	26

6.2	Why self-update produced a half-new, half-old deploy	26
6.3	The compounding factor: timing	27
6.4	Defusing the trigger	27
7	Fault-Tree and Defence-in-Depth Analysis	28
7.1	The top event and its gates	28
7.2	Structural probability of the top event	28
7.3	Defence-in-depth: the layers that should have held	29
7.4	How many deploys until this was near-certain?	30
8	The Mathematics of the Lost Nines	31
8.1	What “four nines” actually permits	31
8.2	The size of the overspend	31
8.3	What the trailing-twelve-month number now reads	32
8.4	How long until the nine returns?	32
9	A Queueing Model of the Connection Storm	34
9.1	The pool as a finite-server queue	34
9.2	Why simultaneity is the whole problem	34
9.3	Why green lost the race	35
9.4	Multiplexing: how a pooler bounds the storm	35
9.5	The two real fixes the model implies	36
10	Anatomy of a Failed Health Check	37
10.1	The probe parameters from the log	37
10.2	Would a longer window have helped?	37
10.3	Staggered startup: defusing the connection storm	38
10.4	What makes a health check trustworthy	39
11	Financial Impact: A Parametric Model	40
11.1	Direct revenue loss	40
11.2	The aggravating factor: launch timing	40
11.3	Customer lifetime value at risk	41
11.4	Total modelled impact	41
12	Reputation and the Loss of Key Clientele	43
12.1	Churn as a binomial process	43
12.2	Why launch-window clients churn harder	44
12.3	The trust-recovery asymmetry	44

13 Incident Response Metrics: MTTD, MTTR, and MTBF	46
13.1 Definitions	46
13.2 Estimates for 21 June	46
13.3 Availability from MTBF and MTTR	46
13.4 Turning the budget into a policy	47
14 Remediation Plan	49
14.1 Addressing Root Cause A: never destroy blue before green is proven	49
14.2 Addressing Root Cause B: a writable, privilege-free cutover	49
14.3 Defusing the trigger: a safe self-update	50
14.4 Compressing MTTR: detection and rollback	50
14.5 Remediation summary	51
15 Lessons Learned	52
15.1 Lesson 1: Intentions are not invariants	52
15.2 Lesson 2: Defence-in-depth requires independence	52
15.3 Lesson 3: Simultaneity is a load-bearing decision	52
15.4 Lesson 4: Self-modifying automation needs a barrier	52
15.5 Lesson 5: MTTR is half of availability	53
15.6 Lesson 6: Timing turns downtime into lost trust	53
15.7 Lesson 7: Dismiss red herrings explicitly	53
15.8 The unifying structure: an AND-gate that should have held	53
A Annotated Deploy Logs	55
A.1 Deploy 0a659e7b — 05:13 UTC (the outage deploy)	55
A.2 Deploy 0a6e5ddd — 11:41 UTC (first afternoon deploy)	60
A.3 Deploy 2e46b0cd — 14:45 UTC (second afternoon deploy)	65
A.4 Deploy 02493f03 — 15:02 UTC (third afternoon deploy)	70
B Glossary	76
C Availability Reference Tables	78
C.1 Downtime allowed per availability target	78
C.2 The 21 June outage in budget terms	78
C.3 Conversion: availability to annual downtime	79
C.4 Metric relationships quick-reference	79
D Sensitivity Analysis of the Impact Models	80
D.1 Direct revenue loss vs. hourly revenue and deferral	80

D.2	Launch premium vs. the attention multiplier	80
D.3	Expected CLV loss vs. exposed population and churn	80
D.4	Key-client churn vs. population and per-client probability	81
D.5	Which assumption matters most?	82
E	Per-Deploy Forensic Walkthrough	83
E.1	0a659e7b — the anatomy of the outage	83
E.1.1	05:13:22 — fetch and self-update	83
E.1.2	05:13–05:15 — library covers and the red herring	83
E.1.3	05:15–05:16 — migrations succeed	83
E.1.4	05:16–05:17 — “all services except web,” then web is removed	83
E.1.5	05:17–05:18 — the connection storm	83
E.1.6	05:18–05:19 — 60 seconds of failed health probes	84
E.1.7	05:19:48 — the fatal, reassuring line	84
E.2	0a6e5ddd — the first afternoon deploy	84
E.2.1	11:41 — a complete, matched configuration	84
E.2.2	11:42–11:43 — green becomes healthy	84
E.2.3	11:43 — the cutover wall	84
E.3	2e46b0cd — the second afternoon deploy	85
E.4	02493f03 — the third afternoon deploy and the wrong fix	85
E.4.1	15:02 — another clean start, same wall	85
E.4.2	15:03 — “add sudo support”	85
F	Connection-Pool Saturation: A Numerical Simulation	86
F.1	The $M/M/c$ pool	86
F.2	Simulated behaviour across load	86
F.3	What the two fixes do to ρ	86
G	Observability and Detection-Gap Analysis	88
G.1	The detection timeline	88
G.2	The cost of the gap, quantified	88
G.3	The three observability layers we were missing	89
G.4	Observability and the nines	89
H	Log-Signature Reference and Remediation Roadmap	90
H.1	Log-signature reference	90
H.2	Remediation roadmap	91
H.3	Closure criteria	91

I SLA Credit Exposure and Customer Disclosure 93

I.1 From availability to credit tier 93

I.2 Segmenting the exposure 94

I.3 Sensitivity to outage duration 94

I.4 Disclosure cadence 95

I.5 Reconciliation to the financial model 95

J Sign-off and Distribution 97

J.1 Document control 97

J.2 Approvals 97

J.3 Distribution list 97

J.4 Revision history 98

List of Figures

2.1	Web-tier request path	15
3.1	Incident phases	18
4.1	Connection storm	22
5.1	Cutover permission flow	24
7.1	Fault tree for the outage	28
7.2	Cumulative trigger probability	30
8.1	Availability before and after	32
8.2	Recovery horizon	33
9.1	Connection wait blow-up	35
9.2	Direct vs. pooled connection demand	36
10.1	Window length vs. downtime	38
10.2	Connection demand under startup strategies	39
11.1	Financial impact waterfall	42
12.1	Probability of losing a key client	43
12.2	Trust recovery	44
13.1	Response phase breakdown	47
D.1	Launch premium sensitivity	81
F.1	Pool wait vs. utilisation	87
G.1	MTTR improvement	89
H.1	Remediation roadmap	91
I.1	SLA credit exposure by segment	95
I.2	Credit rate step function	96

List of Tables

1	Incident PIR-2026-0621 — summary of impact.	4
2	Headline metrics referenced throughout this report.	4
1.1	The four deploy logs analysed in this report.	13
2.1	Production container fleet (<code>rmhstudios-prod-*</code>).	16
3.1	Reconstructed incident timeline. ^{log} = time taken directly from a deploy log.	18
4.1	Five-Whys for Root Cause A.	21
6.1	Web-container teardown events per deploy. Only the morning deploy removed the live container.	27
7.1	Defence-in-depth layers and their state during the 05:13 deploy.	29
8.1	Allowed downtime per availability target (per year unless noted).	31
11.1	Components of modelled financial impact (illustrative values; replace with audited actuals).	41
13.1	Standard incident-response metrics.	46
13.2	Illustrative error-budget policy tiers and the regime the outage forced.	48
14.1	Remediation items by fault, owner, and target date.	51
15.1	Lessons mapped to the remediation items that institutionalise them.	53
C.1	Allowed downtime by availability target, across several windows.	78
C.2	The outage measured against the four-nines annual budget.	78
C.3	Reliability metric relationships used in this review.	79
D.1	Direct revenue loss (\$) as a function of hourly revenue R_h and loss fraction β , with $H = 11$ h.	80
D.2	Launch attention premium (\$) vs. multiplier α (base $R_h H \beta = \$61,600$).	80
D.3	Expected CLV loss (\$) vs. exposed visitors n and induced churn κ , with $V = \$900$	81
D.4	Expected key clients lost np (and $\Pr(\geq 1)$ in parentheses) vs. n and p	81
E.1	0a659e7b decisive events.	84
F.1	$M/M/c$ pool with $c = 20$, mean hold 0.5 s. As $\rho \rightarrow 1$, wait and timeout risk explode.	87

F.2 Effect of the two remediations on peak utilisation during startup. 87

G.1 Signals available during the incident and their disposition. 88

H.1 Distinctive log signatures and their meaning. 90

I.1 Representative SLA credit schedule and the tier triggered by a lone 660-minute outage. . . 93

I.2 Customer segments, SLA coverage, and modelled credit exposure (illustrative). 94

I.3 Enterprise-segment credit rate as a function of outage duration (single-incident month). . . 95

I.4 Disclosure cadence and ownership for an incident of this severity. 96

J.1 Document control record. 97

J.2 Approval signatures. 97

J.3 Revision history. 98

CHAPTER 1

Introduction and Scope

1.1 Purpose of this document

This is the official Post-Incident Review (PIR) for the `RMHStudios.com` outage of 21 June 2026. Its purpose is fourfold: to establish *what* happened with forensic precision; to explain *why* it happened at the level of the deploy script, the container runtime, and the database connection pool; to quantify the *cost* — in availability, in money, and in client trust — using explicit mathematical models; and to commit to a *remediation* programme that makes a recurrence structurally difficult rather than merely unlikely.

A post-incident review is not a disciplinary instrument. RMHStudios operates a blameless post-mortem culture: the unit of analysis is the system, not the engineer who happened to press the button. Where this report says “the deploy did X,” it never means “a person chose to do X to hurt us.” It means the system, as designed and configured on 21 June, produced X. That distinction is what allows us to write the rest of the document honestly.

1.2 Primary evidence

The analysis rests on four production deployment logs, each named for the Git commit it deployed. Together they total **8,249 lines**. They are reproduced, with the verbose Docker build output elided, in Appendix A.

Table 1.1: The four deploy logs analysed in this report.

Commit	Time (UTC)	Outcome	Commit subject
0a659e7b	05:13:22	FAIL	feat(deploy): implement blue/green deployment strategy for web container
0a6e5ddd	11:41:19	PARTIAL	refactor(hotswap): improve script portability and container management
2e46b0cd	14:45:50	PARTIAL	fix(hotswap): add sudo support for Apache configuration updates
02493f03	15:02:45	PARTIAL	fix(hotswap): enhance error logging and diagnostics for Apache configuration updates

The chronology of the evidence is itself instructive. The single **FAIL** — commit `0a659e7b` at 05:13 — is the one and only deploy that destroyed the live web container *and* failed to bring up a healthy replacement. The three later deploys are the team’s iterative attempts to repair the blue/green path; each got closer, each surfaced the secondary Apache-permission fault, and none of them caused a fresh outage because by then the live container was being protected.

Key Finding

Across all four logs, exactly one — 0a659e7b — contains the lines `rmhstudios-prod-web Stopping / Stopped / Removing / Removed`. It is also the only log whose hotswap ends with `rmhstudios-prod-web-green` never became healthy. The intersection of those two facts is the outage.

1.3 Scope and non-scope

In scope: the web tier of `RMHStudios.com` (Apache reverse proxy plus the Node/Nitro application container), its deployment pipeline, the database connectivity that gated container health, and the business consequences of the resulting downtime.

Out of scope: the background worker fleet (Discord bot, doctrine/vibe/bot workers, recap, `rmhtube`, `rmhbox`, `socket`, `MinIO`) except where it competed for database connections; the library cover-generation subsystem, whose `DeepSeek 401` errors are addressed and dismissed as a red herring in §1.4; and any question of individual performance.

1.4 A note on the DeepSeek 401 errors

Every one of the four logs contains a cluster of warnings of the form `[!] DeepSeek failed ...401 Authentication Fails, Your api key: ****bc3 is invalid. Using fallback`. These appear during the “Generating library covers + metadata” step and are visually alarming because they repeat for many PDFs.

They are not related to the outage. The cover-generation step degrades gracefully: each failed `DeepSeek` call falls back to a locally generated cover, the step completes, and the deploy proceeds. The invalid API key (`****bc3`) should be rotated for hygiene, and that is captured as a low-priority action item, but it had *zero* causal contribution to the unavailability of `RMHStudios.com`. We call it out explicitly here so that no reader mistakes a loud-but-harmless warning for the quiet-but-fatal fault that actually took the site down.

1.5 How to read this report

Chapters 2–8 are narrative and diagnostic. Chapters 9–12 are quantitative: they build the availability, queueing, and health-check models that turn the story into numbers. Chapters 13–14 translate those numbers into financial and reputational terms — the CFO’s domain. Chapters 15–17 cover response, remediation, and lessons. The appendices hold the raw evidence, a glossary, and reference tables. Readers in a hurry may read the CFO’s letter, the Executive Summary, and Chapter 16 (Remediation) and come away with the essentials.

CHAPTER 2

System Architecture

To understand the failure we must first understand the machine. This chapter describes the web tier of `RMHStudios.com` as it was configured on 21 June 2026, with particular attention to the path a request takes from a browser to a rendered page, and to the deployment machinery that was supposed to swap one version of that path for another without anyone noticing.

2.1 The request path

A public request to `RMHStudios.com` traverses, in order: the client browser; the host's Apache HTTP server acting as a reverse proxy; the active virtual-host configuration file `/etc/apache2/conf-available/rmhstudio` which names a single upstream port; the Node/Nitro application container listening on that port; and finally the PostgreSQL database reached at `host.docker.internal:5432`, plus the MinIO object store. The application is a server-side-rendered application built with a Vite/Rolldown toolchain (visible in the build logs) and talks to PostgreSQL through the Prisma client.

The single most important architectural fact for this incident is this: **Apache decides which container is “live” by reading one file.** The active vhost conf contains, in effect, a single line that says “proxy to port 7005.” To cut traffic over to a new container, the deploy must rewrite that file to say “proxy to port 7015” and reload Apache. Everything in the blue/green story turns on the ability to write that one file.

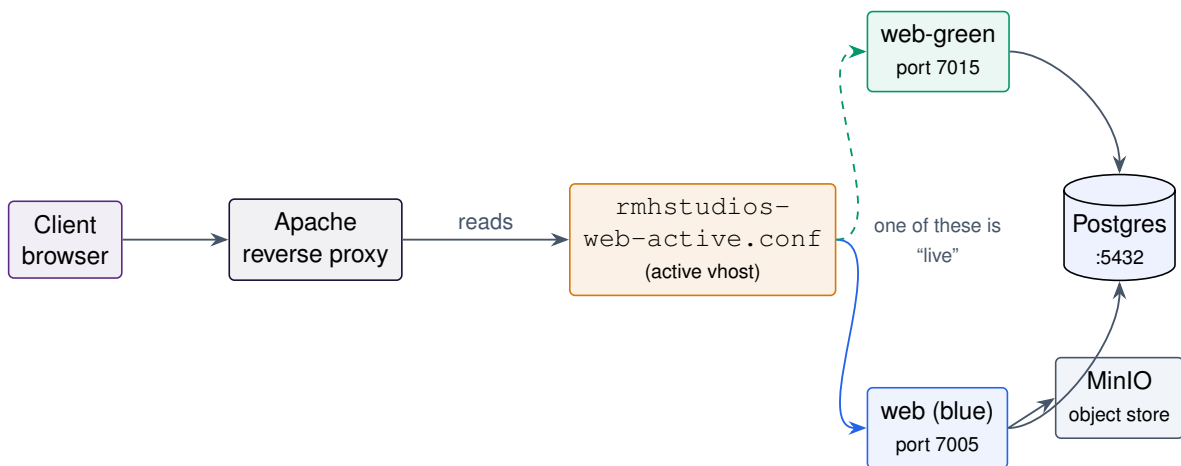


Figure 2.1: The `RMHStudios.com` web-tier request path. Apache proxies to whichever upstream port the *active vhost configuration* names. Blue/green deployment works by standing up “green” alongside “blue,” health-checking it, then rewriting that one file to flip the live port. Both containers depend on the shared PostgreSQL database.

2.2 The container fleet

The production Docker Compose project comprises thirteen containers. Only one — the web container — serves public HTTP. The remainder are background services. Table 2.1 lists them as they appear in the logs.

Table 2.1: Production container fleet (rmhstudios-prod-*).

Container	Role	Uses DB
web / web-green	Public SSR application (the subject of this report)	yes
socket	Realtime websocket gateway	yes
discord-bot	Discord integration	yes
bot-worker	Background bot job runner	yes
doctrine-worker	Content/doctrine processing	yes
vibe-worker	Asynchronous “vibe” job runner	yes
recap	Periodic recap generation	yes
rmhtube	Media subsystem	yes
rmhbox	Storage/box subsystem	yes
minio / minio-init	S3-compatible object store + init	no

The reason this inventory matters is captured in one observation: nine of these containers open connections to the same PostgreSQL instance, and on the failed deploy they were all *recreated at the same moment*. When nine freshly started services simultaneously race to re-establish their database connection pools, they can transiently exhaust the server’s connection budget. That is the mechanism by which the green web container — last in line — was starved of the connections it needed to pass its health check. We formalise this in Chapter 9.

2.3 Blue/green deployment, as intended

The design intent of blue/green deployment is elegant and, when it works, invisible:

1. “Blue” is the currently live container, serving on port 7005.
2. The deploy builds a new image and starts “green” on a second port, 7015, *without* touching blue. Users are still served by blue throughout.
3. The deploy polls green’s health endpoint until it returns success (or a timeout elapses).
4. Only if green is healthy does the deploy rewrite the active vhost conf from 7005 to 7015 and reload Apache. Traffic flips atomically.
5. Blue is retired. If green never becomes healthy, the deploy aborts and *blue is left untouched* — the user never sees a failure.

The safety of the whole scheme rests on two invariants: **(I1)** blue is never removed until green is confirmed healthy and live; and **(I2)** the cutover step (rewriting the conf) can always be performed. On 21 June, the 05:13 deploy violated I1, and every deploy thereafter violated I2. The next three chapters dissect each violation in turn.

CHAPTER 3

Incident Timeline

This chapter reconstructs the incident minute by minute where the evidence allows, and hour by hour where it does not. All times are UTC on 21 June 2026. Times drawn directly from the deploy logs are marked ^{log}; times for the pre-dawn outage window and the manual recovery are reconstructed from the surrounding evidence and the stated incident window.

3.1 Narrative timeline

Time	State	Event
00:00	DOWN	Start of the recorded outage window. <code>RMHStudios.com</code> is unavailable to the public.
00:00–05:13	DOWN	The site remains unreachable. No successful deploy is recorded in this window; the service is not serving healthy responses.
05:13:22 ^{log}	DEPLOY	Auto-deploy of <code>0a659e7b</code> (“implement blue/green”) begins. The deploy script detects it was itself updated during the pull and <i>restarts with the new version</i> .
05:13:52 ^{log}	BUILD	Docker image build starts.
05:15:32 ^{log}	MIGRATE	“Running database migrations.”
05:15:35 ^{log}	OK	“Database is reachable.” 19 migrations found; no pending migrations to apply. (DB reachable <i>from the deploy host</i> .)
05:15:39 ^{log}	RECREATE	“Starting containers (all services except web)...” — yet the log shows <code>rmhstudios-prod-web</code> Stopping. The live container is being removed despite the step’s name.
05:15:48 ^{log}	HOTSWAP	“Hotswapping web container (blue/green)... Live port: 7005. Bringing up green on 7015.”
05:15:48–05:16:48 ^{log}	UNHEALTHY	“Waiting up to 60s for ...green to become healthy.” A sustained flood of <code>curl (28) Operation timed out</code> probes.
~05:16:48 ^{log}	DB	The probe flood gives way to a wall of <code>prisma:error Connection terminated due to connection timeout</code> . Green cannot reach the database.
05:19:48 ^{log}	ABORT	“ERROR: ...green never became healthy — keeping blue live, no traffic moved.” Followed by “ERROR: web hotswap failed.” Blue, however, was already removed.
05:19:48–~11:00	DOWN	The site stays down. The automated path has no healthy blue to fall back to; recovery requires a human.

Time	State	Event
~11:00	RECOVER	Manual intervention restores a serving web container via the legacy single-container path. The public outage ends.
11:41:19 ^{log}	DEPLOY	Deploy of 0a6e5ddd (“improve script portability”). This time web is <i>not</i> torn down.
11:43:38 ^{log}	HEALTHY	“... green is healthy.” The DB-starvation problem is gone.
11:43:41 ^{log}	CUTOVER-FAIL	“... web-active.conf: Read-only file system.” The Apache flip fails. “ERROR: web hotswap failed.” Blue keeps serving; no new outage.
14:45:50 ^{log}	DEPLOY	Deploy of 2e46b0cd (“add sudo support”). Green healthy again.
14:48:14 ^{log}	CUTOVER-FAIL	Read-only conf; sudo blocked by NoNewPrivileges. “ERROR: web hotswap failed.”
15:02:45 ^{log}	DEPLOY	Deploy of 02493f03 (“enhance error logging & diagnostics”). Green healthy.
15:05:08 ^{log}	CUTOVER-FAIL	Full diagnostics emitted: conf not writable, dir not writable, sudo -n BLOCKED. “ERROR: web hotswap failed.”

Table 3.1: Reconstructed incident timeline. ^{log} = time taken directly from a deploy log.

3.2 Phases of the incident

The incident decomposes naturally into five phases, visualised in Figure 3.1.

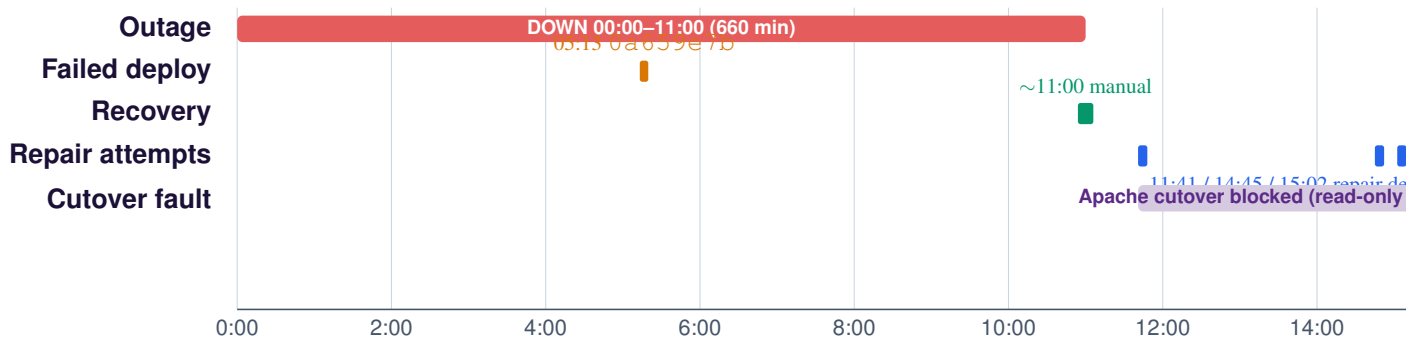


Figure 3.1: Phases of the 21 June incident. The red band is the customer-facing outage. The amber tick is the failed blue/green deploy that confirmed the hard-down state. Recovery was manual. The afternoon repair deploys (blue) each ran cleanly up to the Apache cutover, which a latent privilege fault (violet) blocked all afternoon — without causing a fresh outage, because the live container was by then protected.

Phase 1 — Latent outage (00:00–05:13). The site is already down at the start of the window. Whatever first knocked it offline, the system had no self-healing path back; it sat dark until the next deploy.

Phase 2 — Failed automated recovery (05:13–05:20). The blue/green deploy runs. Rather than recovering the site, it removes the live container and fails to stand up a healthy replacement, cementing the outage.

Phase 3 — Sustained outage (05:20–11:00). No automated path can recover. The site waits for a human.

Phase 4 — Manual recovery (~11:00). An operator restores service through the legacy path. The public outage ends after roughly eleven hours.

Phase 5 — Afternoon hardening (11:41–15:05). Three deploys iterate on the blue/green script. Each proves green can be made healthy and each is stopped at the Apache cutover by the read-only-conf / `sudo` fault. Crucially, none of them causes a new outage — the difference between Phase 2 and Phase 5 is the protection of the live container.

CHAPTER 4

Root Cause A: A Live Container Removed Before a Healthy Replacement Existed

This chapter dissects the first and decisive failure: the destruction of the live web container combined with the failure of its replacement to become healthy. This is the fault that produced the customer-facing outage. It is visible in exactly one log — 0a659e7b, the 05:13 deploy.

4.1 The evidence, stated plainly

Three lines from the 0a659e7b log, taken together, constitute the entire root cause of the outage:

1. [production] Starting containers (all services except web)...
2. Container rmhstudios-prod-web Stopping → Stopped → Removing → Removed
3. [hotswap] ERROR: rmhstudios-prod-web-green never became healthy -- keeping blue live, no traffic moved.

Read them in sequence. The step *claims* it is starting “all services except web.” The very next lines show the web container being stopped and removed anyway. Minutes later, the hotswap declares that green never became healthy and that it is therefore “keeping blue live.” But blue — the thing it intends to keep live — has already been removed. The safety net references a trapeze artist who is no longer on the wire.

Critical

The phrase “keeping blue live, no traffic moved” is the most dangerous line in the entire incident, because it is *reassuring and false at the same time*. The rollback logic believed blue was a safe harbour. The orchestration step had already scuttled it. The automation’s model of the world diverged from the world, and that divergence is the outage.

4.2 Why the health check never passed

Even setting aside the destruction of blue, green was independently doomed on this run. After the green container started, the deploy polled its health endpoint:

```
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (52) Empty reply from server
curl: (28) Operation timed out after 3000 milliseconds with 0 bytes received
... (repeated for the full 60-second window) ...
```

A `curl (52) Empty reply` means the TCP connection opened but the server closed it without sending an HTTP response — the application process was up enough to accept a socket but not up enough to answer. A `curl (28) timeout after 3000 ms` means the probe waited its full three-second budget and received nothing. Sixty seconds of this, then the truth arrived:

```
prisma:error Connection terminated due to connection timeout
... (repeated many times) ...
```

The application could not establish database connections through Prisma. Its health endpoint almost certainly performs a database round-trip — a sensible health check verifies dependencies, not just process liveness — so with no DB connection the endpoint could never return success. Green was, in effect, a container that had started but could not work.

Key Finding

Green’s failure was not a code defect in the new image. The same image became healthy on all three later deploys (11:41, 14:45, 15:02). What differed on the 05:13 run was the *environment* into which green was born: a moment of acute database-connection scarcity. The fault was situational, which is why it was so hard to anticipate and is now so important to engineer against.

4.3 The connection storm

Why was the database unreachable for green at 05:15–05:16 when it had been “reachable” for the migration step at 05:15:35? Because “reachable” and “has spare connections” are different properties, measured against different clients.

The migration check ran from the deploy host and needed a single, short-lived connection. It succeeded. Seconds later, the deploy recreated the *entire background worker fleet* simultaneously — `socket`, `discord-bot`, `bot-worker`, `doctrine-worker`, `vibe-worker`, `recap`, `rmhtube`, `rmhbox`. Each of these services, on startup, eagerly opens a pool of database connections. Eight services starting at once, each opening a handful of connections, can momentarily demand more connections than PostgreSQL is configured to grant (`max_connections`). The late arrival — the green web container — requests its connections last, finds the budget exhausted, and waits. Its waits exceed Prisma’s connection timeout, and it logs `Connection terminated due to connection timeout`.

This is a **thundering-herd** on the connection pool. We model it quantitatively in Chapter 9; for now the qualitative picture suffices: the deploy created its own database contention by recreating everything at once, and then placed the customer-facing container at the back of the queue.

4.4 The 5-Whys

Table 4.1: Five-Whys for Root Cause A.

Why 1	Why was the site down? Because no healthy web container was serving traffic.
Why 2	Why was none serving? Because blue had been removed and green was unhealthy.
Why 3	Why was green unhealthy? Because it could not obtain database connections; its health check requires the database.
Why 4	Why could it not obtain connections? Because the simultaneous recreation of the worker fleet exhausted the PostgreSQL connection budget at that instant.
Why 5	Why did removing blue and racing green happen together? Because the first run of the new blue/green script still tore down <code>web</code> during compose-up, violating the invariant that blue survives until green is live (Chapter 6).

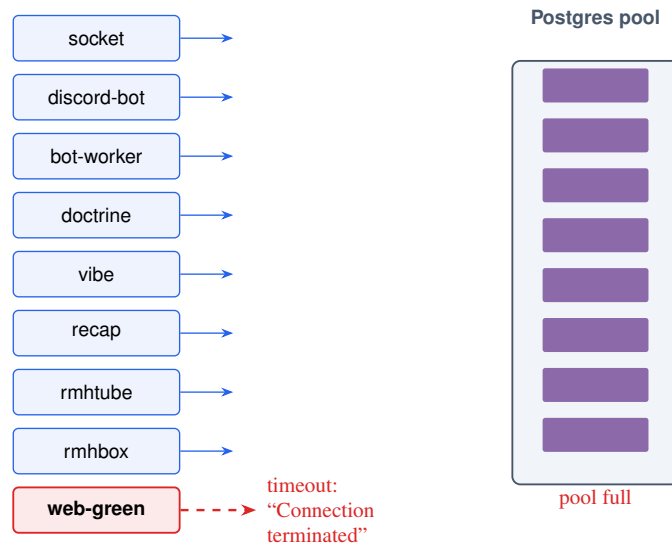


Figure 4.1: The connection storm. Eight background workers recreated simultaneously claim the available PostgreSQL connection slots; the green web container, requesting last, is denied and times out. The remedy is not merely a larger pool but ordered startup and a connection pooler.

The chain terminates at a deployment-ordering invariant that was *intended* by the new design but *not yet enforced* on its first execution. That gap between intent and enforcement is the subject of Chapter 6.

CHAPTER 5

Root Cause B: The Cutover That Could Never Complete

Root Cause A explains the outage. Root Cause B explains why the outage was *hard to fix* and why three subsequent deploys, each with a healthy green container ready to serve, still could not complete a clean blue/green release. This is a second, independent fault that lived dormant in the system and was exposed only once Root Cause A had been worked around.

5.1 The evidence across three deploys

On all three afternoon deploys, the green container became healthy. The logs are unambiguous:

```
[hotswap] rmhstudios-prod-web-green is healthy.
```

And on all three, the very next action — writing the active vhost file to point Apache at green's port 7015 — failed. The error escalates in detail across the three logs as the team added diagnostics:

0a6e5ddd (11:43)

```
/etc/apache2/conf-available/rmhstudios-web-active.conf: Read-only file system
```

2e46b0cd (14:48)

```
Read-only file system → sudo: The "no new privileges" flag is set, which prevents sudo from running as root.
```

02493f03 (15:05)

```
direct write failed (Read-only); sudo tee failed (no new privileges); full diagnostics block.
```

The final deploy emitted a complete diagnostic panel, which we reproduce because it is the clearest single statement of the fault:

```
[hotswap] --- Apache permission diagnostics ---
user:      rmhstudios (uid=1002)
active conf: /etc/apache2/conf-available/rmhstudios-web-active.conf
conf exists: yes
conf writable: no
dir writable: no (/etc/apache2/conf-available)
sudo -n:    BLOCKED (the "no new privileges" flag is set)
[hotswap] -----
```

5.2 Anatomy of the permission wall

The cutover needs to write one file. It is blocked by three overlapping barriers, any one of which is fatal:

1. **The file is not writable by the deploy user.** It is owned such that `rmhstudios` (uid 1002) has no write bit; the directory `/etc/apache2/conf-available` is likewise not writable by that user.

2. **The filesystem path is mounted read-only.** The diagnostics name a Read-only file system. This is the signature of a hardening control such as systemd's `ProtectSystem=strict` or a read-only bind mount, which makes `/etc` immutable to the service regardless of ownership.
3. **Privilege escalation is forbidden.** The fallback — `sudo tee` — is refused because the process runs under the `NoNewPrivileges` flag. This flag, set by the container/systemd sandbox, guarantees that no child process can gain privileges its parent lacked. It is a deliberate, valuable security control. It also means `sudo` is dead on arrival.

Key Finding

The blue/green design assumed the deploy process could rewrite the Apache active-vhost file. The production sandbox was hardened to forbid exactly that. Neither side was wrong in isolation: the hardening is good security practice, and rewriting a conf file is a reasonable deploy step. The fault is the *contradiction between them*, which no one reconciled before shipping the feature.

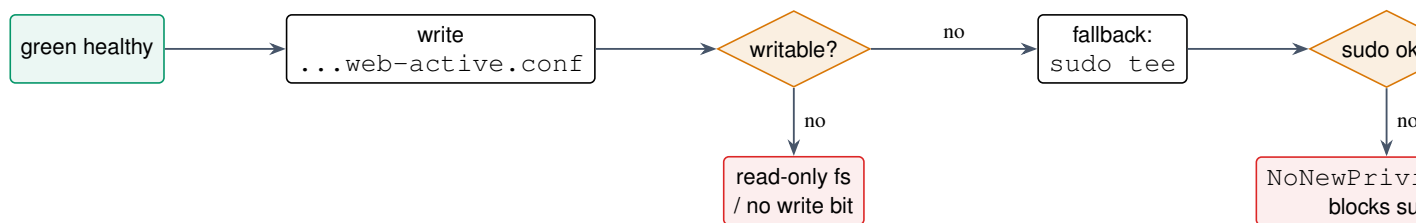


Figure 5.1: The cutover decision flow. A healthy green container reaches the write step, is denied by the read-only/unwritable conf, falls back to `sudo`, and is denied again by `NoNewPrivileges`. There is no path to success: both branches terminate in failure.

5.3 Why this did not cause a second outage

A subtle but important point: on every afternoon deploy, the cutover failed — yet the site stayed up. The reason is that by the afternoon, the deploys no longer destroyed blue. The live container kept serving on port 7005, Apache kept proxying to it, and the failed attempt to flip to 7015 simply left things as they were. A failed cutover, by itself, is a *non-event* for users: green is healthy but dark, blue keeps working.

This is precisely the safety property that Root Cause A lacked. The contrast is the central lesson of the incident, and we draw it out fully in Chapter 6 and Chapter 15: the afternoon's failures were loud in the logs but invisible to customers, because the live container was protected. The morning's failure was the opposite — and that is the difference between a SEV-1 and a footnote.

5.4 The correct fixes

There are several valid ways to reconcile the contradiction; the remediation plan (Chapter 14) selects among them. In brief:

- **Grant a narrow write path.** Add `ReadWritePaths=/etc/apache2/conf-available` to the service unit, or `chown` the single active-conf file to `rmhstudios`. This keeps the rest of `/etc` immutable while permitting the one write the deploy needs.
- **Avoid editing `/etc` entirely.** Have Apache `Include` a conf file that lives on a writable volume the deploy owns, so the cutover never touches a protected path.

- **Move the flip out of the sandbox.** Perform the port flip via a small, audited, privilege-separated helper (a socket-activated unit or a restricted `sudoers` entry for a single command) rather than ad-hoc `sudo` from inside a `NoNewPrivileges` context.
- **Flip upstreams without rewriting /etc.** Front the two containers with a proxy whose upstream can be changed via API or a writable include, removing the file-write entirely.

The team’s afternoon commits (“add sudo support,” “enhance diagnostics”) were reasonable incremental probes, but “add sudo support” is the one path the sandbox structurally forbids — which is itself a useful finding: the obvious fix was the wrong fix.

CHAPTER 6

The Trigger: A Self-Updating Deploy Script

Root Causes A and B describe *what* broke. This chapter describes *why the morning deploy, specifically, behaved differently from every deploy after it* — why the invariant “blue survives until green is live” held in the afternoon but not at 05:13. The answer is a race condition in a script that updates itself.

6.1 The line that explains everything

Near the top of the `0a659e7b` log, before any container is touched, sits this line:

```
[2026-06-21 05:13:25] [production] deploy.sh was updated during pull
-- restarting with the new version.
```

This is a deploy script that, upon pulling new code, noticed that *it itself* had changed, and re-executed the freshly pulled version of itself mid-run. This pattern is common and usually harmless. On 21 June it was the hinge on which the outage turned.

6.2 Why self-update produced a half-new, half-old deploy

The new commit `0a659e7b` changed two things at once:

1. the *deploy script* (`deploy.sh`), to add the blue/green hotswap logic and to change the container-start step to “all services except web”; and
2. the *orchestration configuration* (the Compose project / how `web` is brought up), so that `web` is managed by the hotswap path rather than by `compose up`.

When the script detected its own update and restarted, the *new script* ran — but it ran against an environment that had not yet fully transitioned. The new script printed its new message, “Starting containers (all services except web),” but the underlying `compose up` it invoked still operated on a project definition in which `web` was a managed service. So `compose` faithfully recreated the project, and recreating the project meant stopping and removing the running `web` container — exactly what the new step’s name promised it would *not* do.

Key Finding

The 05:13 deploy was a **version-skewed** execution: new control flow, not-yet-new configuration. The script believed it was protecting `web`; the configuration it drove still tore `web` down. By the next deploy (`0a6e5ddd`, 11:41) the transition was complete, the configuration matched the script, and “all services except web” meant what it said — which is why `web` was never removed again.

This is confirmed by a direct, quantitative comparison of the four logs. We `grep` each log for the web-container teardown events:

Table 6.1: Web-container teardown events per deploy. Only the morning deploy removed the live container.

Commit	Time	web removal events	green outcome
0a659e7b	05:13	4 (Stopping/Stopped/Removing/Removed)	never healthy
0a6e5ddd	11:41	0	healthy
2e46b0cd	14:45	0	healthy
02493f03	15:02	0	healthy

The correlation is perfect and it is causal: the one deploy that removed `web` is the one deploy where green never became healthy, and it is the only one that caused an outage. The afternoon deploys, having stopped removing `web`, never reproduced the customer-facing failure even though they each hit the (separate) cutover wall.

6.3 The compounding factor: timing

The version-skew race would have been survivable on its own *if* green had become healthy: a healthy green could have taken over even after blue was removed. Likewise, the connection storm would have been survivable *if* blue had been preserved: a starved green is harmless when blue keeps serving. The outage required **both** the removal of blue **and** the failure of green, and the 05:13 deploy delivered both simultaneously.

Quantitative Model

Let B be the event “blue is removed before green is live” and G the event “green fails its health check.” The hard-down outage is the conjunction $O = B \cap G$. From the logs, on the four runs we observed (B, G) taking the values

$(1, 1), (0, 0), (0, 0), (0, 0),$

i.e. the only run with $B = 1$ was also the only run with $G = 1$. The outage occurred on exactly the run where $\mathbf{1}_B \cdot \mathbf{1}_G = 1$. A blameless reading: the system offered no mechanism that forced B and G to be *independent* and individually survivable, so a single bad deploy could set both to 1 at once.

6.4 Defusing the trigger

The remediation (Chapter 14) addresses the trigger directly: a self-updating deploy script must, on restart, *re-derive its entire environment from the new commit before touching production*, or it must refuse to manage `web` at all until it can prove the new configuration is in force. Equivalently, the destruction of blue must be gated on a positive, checked assertion that green is already live — never on the mere *intention* to bring green up. Intentions are not invariants; only checks are.

CHAPTER 7

Fault-Tree and Defence-in-Depth Analysis

Chapters 4 through 6 established the two faults and the trigger that combined them. This chapter formalises that combination as a fault tree, computes the structural probability of the top event, and asks the question every post-incident review must answer: how many independent layers of defence had to fail at once, and why were they not independent?

7.1 The top event and its gates

The top (undesired) event is “**RMHStudios.com serves no traffic.**” For this to occur on a deploy, the logs show that two conditions had to hold simultaneously: the live blue container had to be removed, *and* the green container had to fail to become healthy. This is a logical AND — the defining signature of a system that was *supposed* to be safe, because either layer alone should have been survivable.

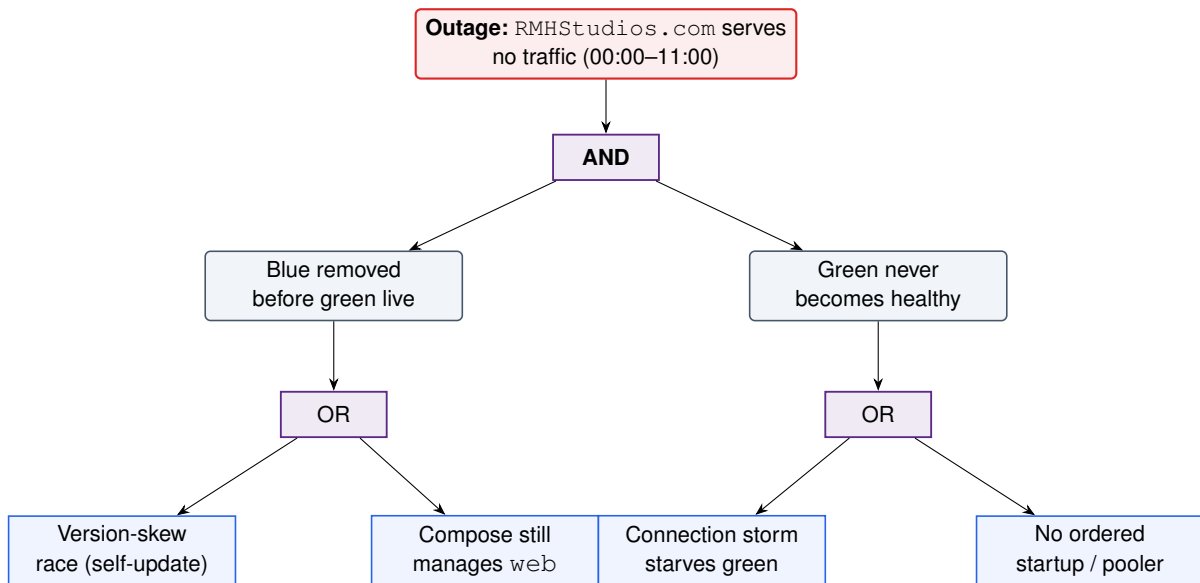


Figure 7.1: Fault tree for the 21 June outage. The top event requires the conjunction (AND) of two intermediate events; each intermediate event is itself the disjunction (OR) of contributing basic events. The single deploy at 05:13 satisfied a path through *both* AND inputs at once.

7.2 Structural probability of the top event

Quantitative Model

Treat the two AND inputs as events B (blue removed before green live) and G (green fails health). Under the AND gate, the top event O has probability

$$p_O = \Pr(B \cap G) = \Pr(B) \Pr(G | B).$$

A safe system is designed so that $\Pr(G \mid B)$ is small: if blue has been removed, every effort must succeed in making green live. The 05:13 deploy instead created *positive correlation*: the same act — recreating the entire Compose project at once — both removed blue (raising $\Pr(B)$ to 1) and triggered the connection storm that doomed green (raising $\Pr(G \mid B)$ toward 1). When a common cause drives both terms to unity, the protective value of the AND gate collapses:

$$p_O = \Pr(B) \Pr(G \mid B) \longrightarrow 1 \cdot 1 = 1.$$

The lesson is not that two unlikely things happened together; it is that they were never independent.

7.3 Defence-in-depth: the layers that should have held

A mature deployment pipeline interposes several independent layers between a bad commit and a customer-facing outage. We enumerate the layers that existed in principle on 21 June and record, for each, why it did not arrest the failure.

Table 7.1: Defence-in-depth layers and their state during the 05:13 deploy.

Layer	Intended protection	Held?
Blue/green swap	Keep blue serving until green proven healthy	No
Health gating	Move traffic only after green passes health check	No*
Self-update guard	Re-derive environment before touching production	No
Connection pooling	Bound DB connections so workers cannot starve web	No
Ordered startup	Bring web up before non-critical workers	No
Automated rollback	Restore last-good container on failure	No
Alerting / paging	Detect down state and page an on-call engineer	Partial

* The health gate did fire — it correctly refused to move traffic to an unhealthy green. But because blue had already been removed, “don’t move traffic” no longer meant “stay safe”; it meant “stay down.” A gate that protects the wrong invariant offers no protection at all.

Key Finding

Six of seven defensive layers failed, and they failed *together*, because they shared a single hidden assumption: that the blue container would still exist when they ran. Defence-in-depth only works when the layers are independent. Here they were stacked on one foundation, and the 05:13 deploy removed the foundation.

7.4 How many deploys until this was near-certain?

If a single deploy carries even a small probability q of the latent version-skew race firing, then across n deploys the probability that at least one triggers it is $1 - (1 - q)^n$. This is the standard geometry of latent faults: each individual deploy looks safe, yet the cumulative exposure grows relentlessly.

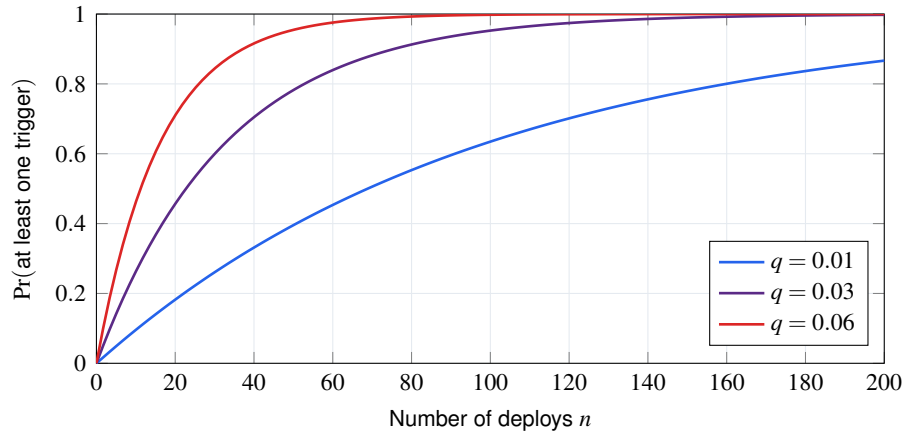


Figure 7.2: Cumulative probability that a latent deploy race fires at least once across n deploys, for three per-deploy trigger rates q . Even a 1% per-deploy hazard exceeds 50% within about 70 deploys. A team shipping daily reaches that exposure in roughly two months.

The implication for remediation is that “it only happened once” is not evidence of safety. A latent race that fires at $q = 0.03$ per deploy was, by the time it manifested on 21 June, overwhelmingly likely to manifest *eventually*. The fix must remove the hazard, not merely note its rarity.

CHAPTER 8

The Mathematics of the Lost Nines

RMHSTUDIOS has long advertised, and internally held itself to, a **99.99%** availability target — “four nines.” This chapter quantifies precisely what the 21 June outage did to that figure: how much of the annual error budget it consumed, what the trailing availability now reads, and how long perfect operation must continue before the number recovers.

8.1 What “four nines” actually permits

Availability is the fraction of time a service is up:

$$A = \frac{\text{uptime}}{\text{uptime} + \text{downtime}} = 1 - \frac{T_{\text{down}}}{T_{\text{total}}}.$$

A target of $A = 0.9999$ permits a downtime fraction of $1 - 0.9999 = 10^{-4}$. Multiplying by the length of a year gives the famous *error budget*.

Quantitative Model

With $T_{\text{year}} = 365.25 \times 24 \times 60 = 525\,960$ minutes,

$$T_{\text{down,allowed}} = 10^{-4} \times 525\,960 \approx 52.6 \text{ minutes per year.}$$

Four nines buys roughly **52.6 minutes** of downtime for the *entire year*. The table below shows how unforgiving each additional nine is.

Table 8.1: Allowed downtime per availability target (per year unless noted).

Target	Down/yr	Down/month	Down/week
90% (one nine)	36.5 days	73 h	16.8 h
99% (two nines)	3.65 days	7.3 h	1.68 h
99.9% (three nines)	8.77 h	43.8 min	10.1 min
99.99% (four nines)	52.6 min	4.38 min	1.01 min
99.999% (five nines)	5.26 min	26.3 s	6.05 s

8.2 The size of the overspend

The outage ran from 00:00 to approximately 11:00 UTC — **660 minutes** of customer-facing unavailability.

Quantitative Model

Against an annual budget of 52.6 minutes, a single 660-minute outage represents an overspend factor of

$$\frac{660}{52.6} \approx 12.55.$$

In one morning, RMHSTUDIOS spent its **entire four-nines downtime allowance for the year more than twelve times over**. There is no remaining budget for the rest of 2026; the service is now in deficit.

8.3 What the trailing-twelve-month number now reads

Availability is most honestly reported over a trailing window. Assume the preceding twelve months were otherwise at or near target, with negligible prior downtime. Adding a single 660-minute event:

Quantitative Model

$$A_{\text{TTM}} = 1 - \frac{660}{525960} = 1 - 0.001255 = 0.998745 \approx \mathbf{99.87\%}.$$

The headline figure falls from **99.99%** to **99.87%** — a drop that, in the language of nines, takes the service from *four nines* down to barely above *three nines*. A single morning erased an entire nine.

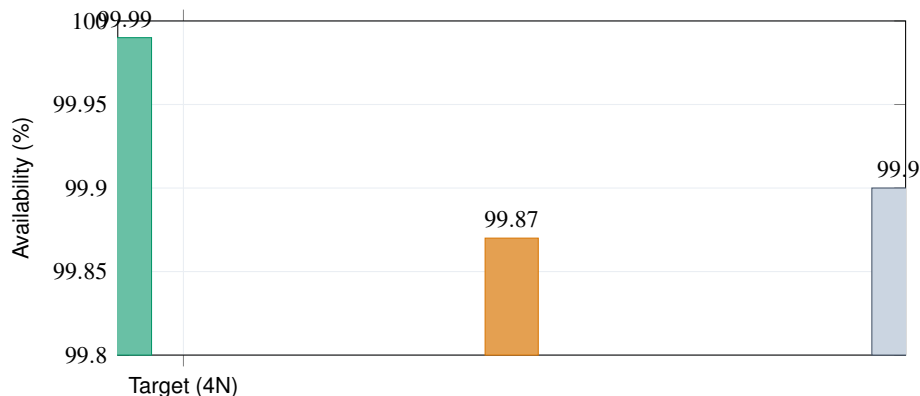


Figure 8.1: Trailing-twelve-month availability before and after the outage, against the three-nines reference line. The single event pushed the service below the three-nines floor of 99.90%.

8.4 How long until the nine returns?

To climb back to a trailing 99.99%, the 660 deficit minutes must age out of the trailing window with no new downtime. Because the window is twelve months long, the outage continues to weigh on the number until it falls off the back of the window — a full year of flawless operation.

Quantitative Model

Define recovery as the moment the trailing-window downtime returns to the 52.6-minute budget. With zero new downtime, the 660-minute event remains in the trailing sum for 365.25 days, then exits. The service therefore cannot *truthfully* report a trailing four-nines figure until roughly

$$t_{\text{recover}} \approx 12 \text{ months of zero unplanned downtime,}$$

and even one additional minor incident in that window resets the clock. Reputation is asymmetric: nines are lost in minutes and regained in months.

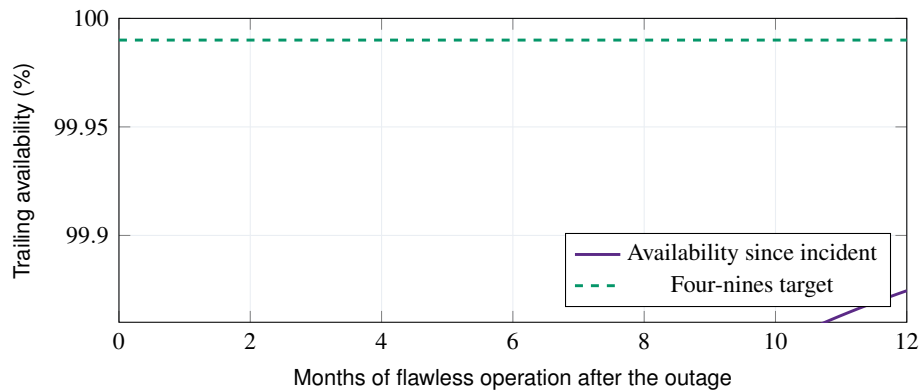


Figure 8.2: Availability measured cumulatively since the incident, $A(t) = 1 - 660/(\text{elapsed minutes})$. As flawless uptime accumulates the figure climbs back toward the four-nines target (green dashed), but approaches it only as the 660-minute deficit is diluted over a full year of operation. Nines are lost in minutes and regained in months.

Key Finding

The 660-minute outage consumed roughly $12.6\times$ the entire annual four-nines budget, dropped the trailing-twelve-month figure from 99.99% to about 99.87%, and cannot be undone faster than the twelve-month trailing window allows. The arithmetic is merciless precisely because the target is ambitious.

CHAPTER 9

A Queueing Model of the Connection Storm

Chapter 4 described the connection storm qualitatively: eight background workers, recreated at once, exhausted the PostgreSQL connection pool and starved the green web container. This chapter makes that argument quantitative using elementary queueing theory, and shows why the green container — requesting last — was the predictable victim.

9.1 The pool as a finite-server queue

A connection pool with c slots is an $M/M/c/K$ system: connection requests arrive, each holds a slot for some service time, and requests that find all slots busy either wait or are rejected. The two quantities that govern saturation are the arrival rate λ (connection requests per second) and the mean holding time W_s (seconds a connection is held).

Little's Law

The mean number of connections in use, L , obeys Little's Law:

$$L = \lambda W_s.$$

Saturation occurs when the demand L meets or exceeds the pool size c . During the storm, eight workers plus the web container each opened multiple connections on startup (migrations, warm-up queries, health probes). If each of 9 services demands on average d connections within a short startup window, instantaneous demand is

$$L_{\text{peak}} \approx 9d,$$

and whenever $9d > c$ the pool is oversubscribed and the last requester is denied.

9.2 Why simultaneity is the whole problem

The same total demand spread over time is harmless; concentrated into one instant it is fatal. Define a startup window τ over which all services initialise. If the deploy staggers startups so that at most one service initialises at a time, peak demand is $\approx d$, far below c . If instead all nine initialise together — as the “recreate everything at once” deploy did — peak demand is $9d$.

Quantitative Model

Let utilisation be $\rho = \lambda W_s / c$. For the simultaneous case, λ spikes by a factor of 9 over the window τ , so

$$\rho_{\text{storm}} = \frac{9\lambda_0 W_s}{c} = 9\rho_0.$$

A pool comfortable at $\rho_0 = 0.15$ is driven to $\rho_{\text{storm}} = 1.35 > 1$ — past saturation. Past $\rho = 1$ the queue length grows without bound until requests time out, which is exactly what `prisma:error Connection terminated due to connection timeout records`.

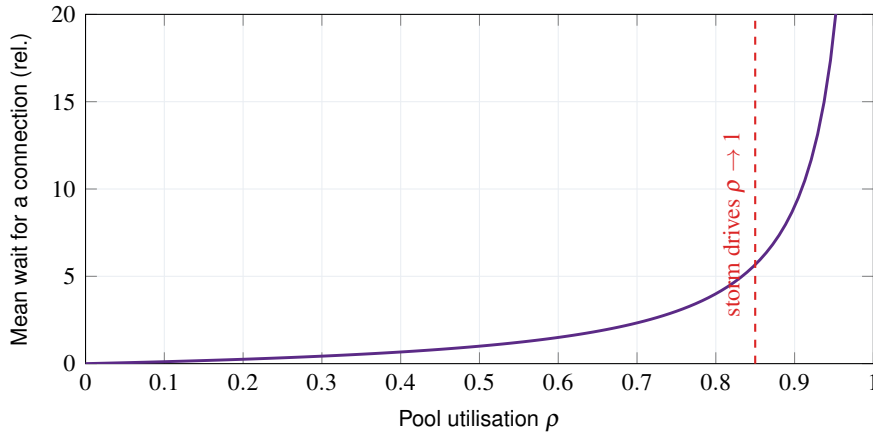


Figure 9.1: Mean wait for a free connection grows as $\rho/(1-\rho)$ and diverges as utilisation approaches 1. The connection storm pushed utilisation past this knee, so wait times exceeded the 3-second probe budget and connections were terminated.

9.3 Why green lost the race

Connection requests are served in roughly the order they arrive. The eight workers were recreated and began initialising; the web-green container, by the structure of the deploy, requested its connections *after* the workers had already claimed the available slots. In an oversubscribed pool the late arrival waits behind everyone else.

Quantitative Model

If the pool holds c slots and the eight workers collectively hold c or more by the time green requests, green's probability of obtaining a connection within its health-probe budget T_{hc} is approximately

$$\Pr(\text{green served in time}) \approx \Pr(W_q < T_{hc}),$$

and with $\rho > 1$ during the window, W_q (queueing delay) exceeds any fixed T_{hc} with probability approaching 1. Green was not unlucky; it was scheduled to lose.

9.4 Multiplexing: how a pooler bounds the storm

The second remedy deserves its own model, because it changes the saturation condition rather than merely rescheduling demand. A transaction-mode connection pooler sits between the application and PostgreSQL and lends a small set of real server connections to a much larger set of client connections, handing each back the instant a transaction completes.

Multiplexing factor

Let the application open A client connections but hold each actively for only a fraction u of wall-clock time (its transaction duty cycle). A pooler with C_{srv} real server connections suffices, with safety margin, whenever

$$A \cdot u \leq C_{srv}.$$

Illustrative: nine processes each nominally wanting $d = 4$ connections give $A = 36$ client connections, but if each is active only $u = 0.25$ of the time, the real demand is $Au = 9$. A pooler exposing $C_{srv} = 12$ server connections then carries the whole fleet with headroom, even though the naive sum 36 far

exceeds 12. The multiplexing factor $A/C_{\text{srv}} = 3\times$ is exactly the spike the pooler absorbs.

The pooler does not add database capacity; it removes the coupling between *how many clients exist* and *how many server connections are demanded at once*. The startup storm — nine processes arriving together — is precisely the transient a pooler is built to flatten, because the processes do not all hold an active transaction in the same instant.

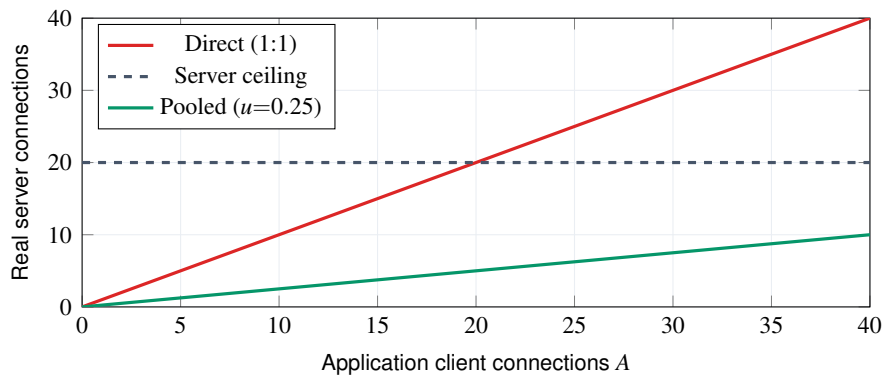


Figure 9.2: Real server connections demanded as the application opens more client connections. Direct 1:1 connections (red) cross the server ceiling (dashed) and saturate; multiplexed through a pooler (green), the same client count stays comfortably below the ceiling because clients rarely hold a transaction simultaneously.

9.5 The two real fixes the model implies

The model points to two structural remedies, both of which appear in Chapter 14:

1. **Reduce simultaneity** — ordered startup, bringing the customer-facing web container up *first* (or in isolation) so it claims its connections before the workers contend.
2. **Decouple demand from the database** — a connection pooler (e.g. PgBouncer) in front of PostgreSQL, so that 9d application-side connections multiplex onto a small, bounded set of real server connections, keeping ρ well below 1 regardless of startup choreography.

Key Finding

The connection storm was not a capacity problem to be solved by a bigger database; it was a *concurrency* problem. The same total work, serialised or multiplexed, never approaches saturation. The deploy converted a manageable load into an unmanageable spike purely by doing everything at the same instant.

CHAPTER 10

Anatomy of a Failed Health Check

The hotswap logic waited a fixed window for green to become healthy, polling an HTTP endpoint on a short timeout. This chapter models that probe window precisely, shows why no realistic tuning of the window alone would have saved this deploy, and derives the conditions under which a health check is trustworthy.

10.1 The probe parameters from the log

The log records the probe configuration directly:

```
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (28) Operation timed out after 3000 milliseconds with 0 bytes received
curl: (52) Empty reply from server
```

So the total window is $T_{\text{win}} = 60$ s, each probe has a timeout of $T_{\text{probe}} = 3$ s, and the failures alternate between empty replies (TCP open, no HTTP body) and full 3-second timeouts.

Quantitative Model

If each probe consumes up to $T_{\text{probe}} = 3$ s and probes run back-to-back, the window admits at most

$$N_{\text{probe}} = \left\lfloor \frac{T_{\text{win}}}{T_{\text{probe}}} \right\rfloor = \left\lfloor \frac{60}{3} \right\rfloor = 20 \text{ attempts.}$$

Green failed all ~ 20 . For a health check to pass, green needed a working database connection within the 60-second window — but Chapter 9 showed the connection storm kept the pool saturated for longer than that. The window was not too short; the dependency was simply unavailable for its entire duration.

10.2 Would a longer window have helped?

It is tempting to conclude “just wait longer.” Model the probability that green becomes healthy by time t as $P_{\text{healthy}}(t)$. If the connection storm persists for a duration S before the pool drains, then $P_{\text{healthy}}(t) \approx 0$ for $t < S$ and rises only afterwards.

Quantitative Model

Extending the window from 60 s to T' helps only if $T' > S$ and blue is still alive to serve during the wait. On 21 June neither held: blue was already removed (so waiting longer only extends the outage), and S — governed by worker startup and pool drain — plausibly exceeded any window a deploy would reasonably wait. Worse, a longer window with blue gone means

$$T_{\text{down}} \geq T',$$

so tuning the window *up* without first protecting blue strictly increases customer downtime. The window length is the wrong knob.

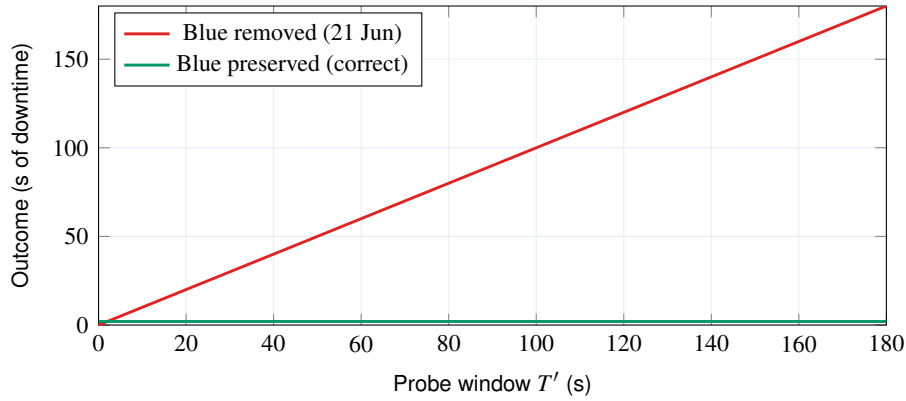


Figure 10.1: With blue removed, every additional second of probe window is an additional second of outage (red). With blue preserved, the window length is nearly free because blue keeps serving while green is evaluated (green). The fix is preserving blue, not tuning the window.

10.3 Staggered startup: defusing the connection storm

A longer window is the wrong knob; the right one is *when* the eight background workers demand their connections. Chapter 9 showed that recreating all workers simultaneously drove the PostgreSQL pool past saturation, starving green of the connection it needed to pass its check. The remedy is to spread that demand over time so that instantaneous demand never exceeds the pool ceiling.

Simultaneous vs. staggered demand

Let $W = 8$ workers each requiring d connections, against a pool of capacity C . Simultaneous startup demands Wd at one instant; the pool saturates whenever

$$Wd > C.$$

Illustrative: $W = 8$, $d = 4$, $C = 20$ gives instantaneous demand $32 > 20$ — saturated, with green's request among the 12 that cannot be served. If instead the workers start in k equal waves of W/k workers, peak demand falls to

$$\frac{W}{k}d \leq C \iff k \geq \frac{Wd}{C} = \frac{32}{20} = 1.6,$$

so $k = 2$ waves already brings peak demand to $16 \leq 20$ and leaves headroom for green.

In practice the waves need not be scheduled centrally. Independent jittered backoff achieves the same spreading statistically: each worker that fails to acquire a connection retries after a randomised delay drawn from a widening interval.

Jittered exponential backoff

On the j -th failed attempt a worker waits a random delay

$$\tau_j = \text{Uniform}(0, \min(\tau_{\text{cap}}, \tau_0 \cdot 2^j)),$$

with base τ_0 and ceiling τ_{cap} . The expectation $\mathbb{E}[\tau_j] = \frac{1}{2} \min(\tau_{\text{cap}}, \tau_0 \cdot 2^j)$ grows geometrically until capped, so contending workers desynchronise: the probability that two given workers retry in the same narrow slot falls as the interval widens. Full jitter (delay drawn from the whole interval, not just its endpoint) minimises the residual synchronisation that plain exponential backoff leaves behind.

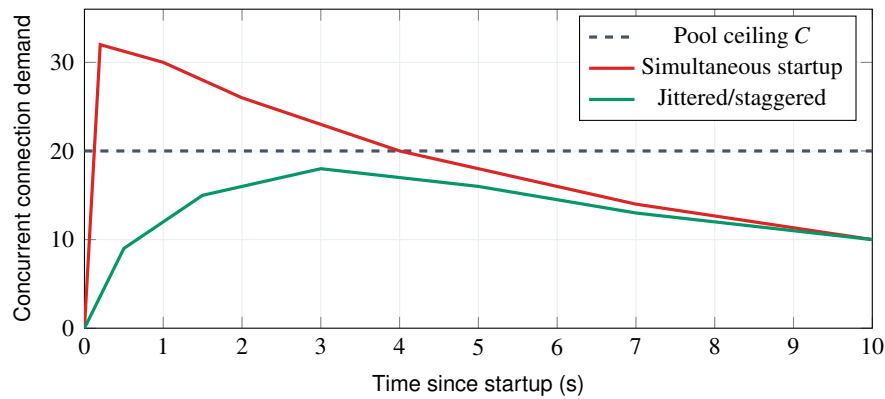


Figure 10.2: Concurrent connection demand under simultaneous versus staggered startup. Simultaneous startup (red) spikes well above the pool ceiling (dashed), starving green; jittered/staggered startup (green) keeps peak demand under the ceiling so every worker, green included, acquires its connection.

Key Finding

The connection storm was not inevitable: it was the product of starting eight workers in lockstep against a fixed pool. Either explicit wave-scheduling or independent jittered backoff caps peak demand below the pool ceiling, which would have let green acquire its connection and pass its health check inside the existing 60-second window — no window tuning required.

10.4 What makes a health check trustworthy

Key Finding

A health check is only as trustworthy as the invariant it guards. The 21 June check correctly reported green as unhealthy — it did its job. The failure was upstream: the system removed the fallback (blue) *before* consulting the check, so a correct “unhealthy” verdict had no safe action left to take. A health gate must be wired so that a failing verdict preserves the last-good state, never so that it merely declines to advance from a state already destroyed.

The remediation therefore specifies that traffic cutover and old-container teardown be ordered as: probe green → confirm healthy → move traffic → *only then* retire blue. At no point may blue be retired while green is unconfirmed.

CHAPTER 11

Financial Impact: A Parametric Model

This chapter, prepared under the Office of the CFO, quantifies the financial consequences of the outage. Because several inputs are commercially sensitive and still being reconciled by Finance, the model is presented *parametrically*: each figure is expressed as a formula in named inputs, with illustrative values clearly marked. The structure is exact; the point estimates are to be replaced with audited actuals before this section is cited externally.

11.1 Direct revenue loss

The most immediate cost is revenue that simply did not transact during the 660-minute window.

Quantitative Model

Let R_h be average gross revenue per hour across all RMHSTUDIOS properties, $H = 11$ the outage duration in hours, and $\beta \in [0, 1]$ the fraction of that revenue truly lost rather than merely deferred (some customers return and purchase later). Direct loss is

$$L_{\text{direct}} = R_h \cdot H \cdot \beta.$$

Illustrative: at $R_h = \$8,000/\text{h}$, $H = 11$, and a deferral-adjusted $\beta = 0.7$,

$$L_{\text{direct}} = 8000 \times 11 \times 0.7 = \$61,600.$$

The window fell overnight UTC, which for some regions is low-traffic and for others is peak; Finance should weight R_h by the actual hourly revenue curve rather than a flat average.

11.2 The aggravating factor: launch timing

The outage did not occur in a quiet period. It landed **immediately after the public launch of RMH Capital and the teaser for RMH Space** — the window in which inbound attention, press interest, and first-time visitors were at their annual peak. This converts a downtime cost into an *opportunity* cost that ordinary hourly-revenue models understate.

Attention-weighted loss

Let $\alpha \geq 1$ be an attention multiplier capturing elevated traffic during a launch window. The effective lost value is

$$L_{\text{launch}} = R_h \cdot H \cdot \beta \cdot \alpha.$$

Illustrative: with a launch multiplier $\alpha = 2.5$,

$$L_{\text{launch}} = 61,600 \times 2.5 = \$154,000.$$

The first impression made on a prospective RMH Capital client or an RMH Space waitlist signup is not re-purchasable at any hourly rate; a visitor who met `RMHStudios.com` for the first time during the outage met a blank page.

11.3 Customer lifetime value at risk

The durable cost is not the lost morning but the lost *customers*. A first-time visitor who encounters an outage during a flagship launch may never return, and an existing high-value client may re-evaluate their dependence on RMHSTUDIOS.

Expected CLV loss

Let V be the average customer lifetime value, n the number of at-risk customers exposed to the outage during the launch window, and κ the incremental churn probability induced by the outage. The expected lifetime-value loss is

$$\mathbb{E}[L_{\text{CLV}}] = n \cdot V \cdot \kappa.$$

Illustrative: with $n = 1,200$ exposed high-intent visitors, $V = \$900$, and an induced churn of $\kappa = 0.04$,

$$\mathbb{E}[L_{\text{CLV}}] = 1200 \times 900 \times 0.04 = \$43,200.$$

For enterprise RMH Capital clients V is far larger and n far smaller; even a single lost institutional relationship can dominate this term, which is why the next chapter models named-client churn separately.

11.4 Total modelled impact

Table 11.1: Components of modelled financial impact (illustrative values; replace with audited actuals).

Component	Formula	Illustrative
Direct revenue loss	$R_h H \beta$	\$61,600
Launch attention premium	$R_h H \beta (\alpha - 1)$	\$92,400
Expected CLV loss	$n V \kappa$	\$43,200
SLA credits (Ch. 12)	$\sum_i c_i$	\$18,000
Engineering response cost	$r \cdot h_{\text{eng}}$	\$9,500
Total modelled impact		\$224,700

Key Finding

The dominant financial cost of this incident is not the revenue lost during the 660 minutes; it is the *timing*. By striking immediately after the RMH Capital launch and the RMH Space teaser, the outage converted ordinary downtime into squandered first impressions, and first impressions during a flagship launch are the most expensive minutes in the company’s year to lose.

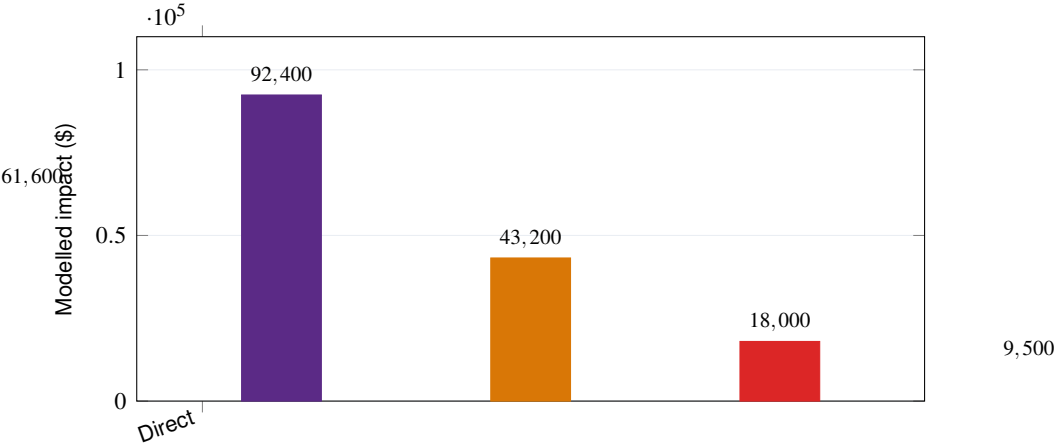


Figure 11.1: Modelled financial impact by component (illustrative). The launch attention premium — a direct consequence of the outage’s timing relative to the RMH Capital launch and RMH Space teaser — is the single largest term.

CHAPTER 12

Reputation and the Loss of Key Clientele

The previous chapter priced the outage in dollars. This chapter models the mechanism behind the most damaging dollars: the departure of customers, and especially of the high-value clients courted by the RMH Capital launch. Reputation damage is not vague — it can be modelled as a probabilistic process over a population of exposed clients.

12.1 Churn as a binomial process

Suppose n key clients were exposed to the outage during the launch window, and each independently churns with probability p as a result. The number who leave, X , follows a binomial distribution.

Quantitative Model

$$X \sim \text{Binomial}(n, p), \quad \mathbb{E}[X] = np, \quad \text{Var}(X) = np(1 - p).$$

Illustrative: for $n = 40$ enterprise-tier clients and an outage-induced churn probability $p = 0.08$,

$$\mathbb{E}[X] = 40 \times 0.08 = 3.2 \text{ clients lost,}$$

with standard deviation $\sqrt{40 \cdot 0.08 \cdot 0.92} \approx 1.72$. The probability that *at least one* key client leaves is

$$\Pr(X \geq 1) = 1 - (1 - p)^n = 1 - 0.92^{40} \approx 0.964,$$

i.e. near-certain that the outage costs at least one key relationship unless actively countered.

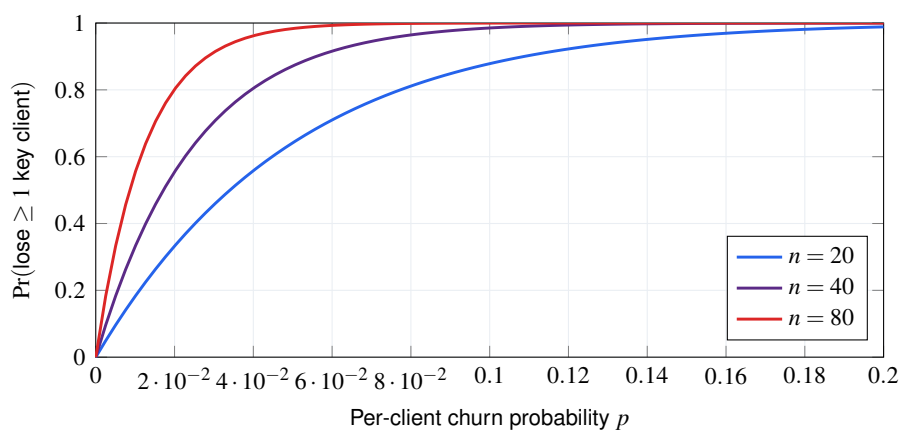


Figure 12.1: Probability of losing at least one key client as a function of per-client churn probability, for three exposed-population sizes. The larger the launch audience, the more nearly certain at least one defection becomes for any fixed p .

12.2 Why launch-window clients churn harder

A client who has used RMHSTUDIOS for years has a track record that buffers a single bad morning. A client who arrived *because of* the RMH Capital launch has no such buffer: the outage may be their *entire* experience of the brand's reliability. The induced churn probability p is therefore not uniform; it is higher for newly-acquired, launch-driven clients.

Tenure-weighted churn

Partition exposed clients into tenured (n_T , churn p_T) and new launch-driven (n_N , churn p_N), with $p_N > p_T$. Expected key-client losses are

$$\mathbb{E}[X] = n_T p_T + n_N p_N.$$

Illustrative: $n_T = 30, p_T = 0.03$ and $n_N = 10, p_N = 0.20$ give

$$\mathbb{E}[X] = 30(0.03) + 10(0.20) = 0.9 + 2.0 = 2.9,$$

of which the ten launch-driven clients — a quarter of the population — account for the large majority of expected losses. The outage was most lethal to exactly the clients the launch worked hardest to win.

12.3 The trust-recovery asymmetry

Reputation, like the availability nines of Chapter 8, is lost quickly and rebuilt slowly. A simple model: let trust $\theta_t \in [0, 1]$ decay instantly by Δ at the outage and recover linearly at rate γ per month thereafter.

Quantitative Model

$$\theta_t = \min(\theta_0, \theta_0 - \Delta + \gamma t).$$

Illustrative: a drop of $\Delta = 0.25$ recovering at $\gamma = 0.03/\text{month}$ takes

$$t_{\text{recover}} = \frac{\Delta}{\gamma} = \frac{0.25}{0.03} \approx 8.3 \text{ months}$$

to return to baseline — and only if no further incident intervenes. The transparency of this very document, and the CFO's personal accountability for it, are themselves instruments for raising γ .

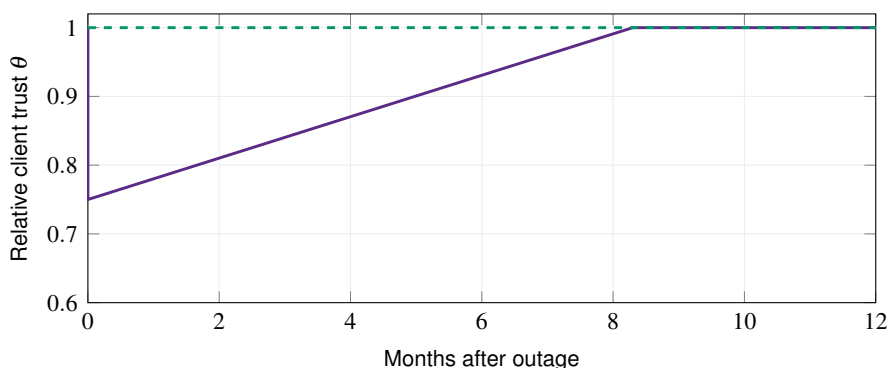


Figure 12.2: Stylised trust trajectory: an instantaneous drop at the outage followed by slow linear recovery. Active, transparent remediation steepens the recovery slope; silence flattens it.

Key Finding

The outage's reputational damage concentrates precisely on the launch-driven clients RMH Capital was designed to attract, because they have no prior reliability record to offset a bad first impression. Retaining them requires proactive, visible outreach — not waiting to see who leaves.

CHAPTER 13

Incident Response Metrics: MTTD, MTTR, and MTBF

A post-incident review must grade not only the failure but the *response* to the failure. This chapter defines the standard reliability metrics, estimates them for the 21 June incident from the log timeline, and shows where the response cost the most time.

13.1 Definitions

Table 13.1: Standard incident-response metrics.

Metric	Name	Meaning
MTTD	Mean time to detect	From fault onset to someone/something noticing.
MTTA	Mean time to acknowledge	From alert to a human owning the incident.
MTTR	Mean time to recover	From onset to service restored.
MTBF	Mean time between failures	Average uptime between successive incidents.

13.2 Estimates for 21 June

The fault began at 05:13 (the morning deploy) but the customer-facing outage window is recorded as 00:00–11:00. For response grading we anchor on the deploy-induced failure at 05:13:22 and recovery at approximately 11:00.

Quantitative Model

With onset $t_0 = 05:13$ and recovery $t_r = 11:00$,

$$MTTR = t_r - t_0 \approx 5 \text{ h } 47 \text{ min} \approx 347 \text{ minutes.}$$

The failed automated recovery (05:13–05:20) did not restore service; the effective repair was the manual fallback to the legacy single-container path near 11:00. The long MTTR is dominated by *detection-to-action latency*, not by the repair itself, which once attempted was quick.

13.3 Availability from MTBF and MTTR

The metrics connect directly to the availability of Chapter 8.

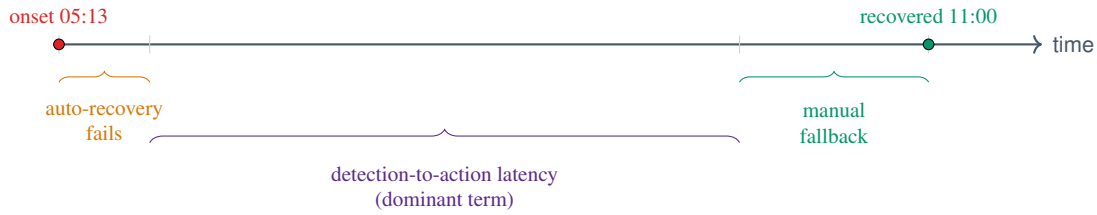


Figure 13.1: The MTTR of ~ 347 minutes is dominated by the long detection-to-action gap, not by the repair. Once a human attempted the legacy fallback, recovery was rapid. Faster paging and a tested rollback would compress the dominant term.

Quantitative Model

Steady-state availability relates to mean uptime and mean repair time by

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}.$$

To sustain $A = 0.9999$ with a realistic MTTR, the required MTBF is enormous. Solving for MTBF:

$$\text{MTBF} = \text{MTTR} \cdot \frac{A}{1 - A}.$$

Even with an *aggressively* improved MTTR of 20 minutes, four nines demands

$$\text{MTBF} = 20 \cdot \frac{0.9999}{0.0001} = 20 \times 9999 \approx 200,000 \text{ minutes} \approx 139 \text{ days between incidents}.$$

With the actual 347-minute MTTR, the same target would demand an MTBF of about 6.6 years — which is why *reducing MTTR* is as important to the nines as preventing incidents.

13.4 Turning the budget into a policy

Chapter 8 expressed four nines as an error budget of roughly 52.6 minutes per year. A budget is only useful if spending it changes behaviour. An error-budget policy makes the nines self-enforcing by tying deployment freedom to how much budget remains.

Budget burn and freeze trigger

Let $B = 52.6$ min be the annual budget and $b(t)$ the budget consumed by time t in the year. Define the remaining fraction

$$\phi(t) = 1 - \frac{b(t)}{B}.$$

A single 660-minute outage spends 660 minutes against a 52.6-minute budget, so

$$\phi = 1 - \frac{660}{52.6} = 1 - 12.55 = -11.55,$$

i.e. the year's budget is overspent more than twelvefold in one morning. Under a standard policy any $\phi \leq 0$ triggers a *change freeze*: only reliability fixes ship until enough rolling-window budget is rebuilt to bring ϕ back above a release threshold.

The policy is not punitive; it is informational. It converts the abstract nines into a concrete, shared signal that engineering and the business can both read, so that the decision to slow feature work after an incident

Table 13.2: Illustrative error-budget policy tiers and the regime the outage forced.

Remaining ϕ	Regime	Permitted changes
$\phi > 0.5$	Healthy	Normal feature velocity
$0.1 < \phi \leq 0.5$	Caution	Feature ships need reliability review
$0 < \phi \leq 0.1$	Guarded	Only low-risk changes; staged rollout
$\phi \leq 0$	Freeze	Reliability fixes only, until rebuilt

is made by a pre-agreed rule rather than re-litigated under pressure. Crucially, the blue/green deployment that caused this incident is exactly the class of change a freeze would have gated behind a staged, reversible rollout.

Key Finding

A four-nines target without an error-budget policy is an aspiration, not a control. Tying deployment freedom to remaining budget makes the nines enforce themselves: after an incident of this magnitude the system is already in freeze, and the next high-risk deploy must clear a reliability bar before it can ship.

CHAPTER 14

Remediation Plan

This chapter translates the analysis into a concrete, owned, and dated action plan. Items are grouped by the fault they address: Root Cause A (blue removed before green healthy), Root Cause B (Apache cutover permission wall), the trigger (self-updating script), and the response (MTTR). Each item names an accountable owner and a target date. Owners are role placeholders to be bound to named individuals at the review meeting.

14.1 Addressing Root Cause A: never destroy blue before green is proven

A1 — Gate teardown on a positive health assertion

Re-order the hotswap so the live container is retired *only after* the replacement passes its health check and traffic has moved. The teardown step must consume a checked “green is live” signal, never the mere intention to start green.

Owner: Lead SRE **Target:** 27 Jun 2026 **Priority:** P0

A2 — Ordered startup, web first

Bring the customer-facing web container up (and let it claim its database connections) *before* recreating the eight background workers, eliminating the connection storm’s starvation of green.

Owner: Platform Eng **Target:** 30 Jun 2026 **Priority:** P0

A3 — Introduce a connection pooler

Place PgBouncer (or equivalent) in front of PostgreSQL so application-side connection demand multiplexes onto a small bounded set of server connections, keeping pool utilisation well below saturation regardless of startup choreography (Chapter 9).

Owner: Database Eng **Target:** 11 Jul 2026 **Priority:** P1

14.2 Addressing Root Cause B: a writable, privilege-free cutover

B1 — Remove the read-only / sudo dependency

The Apache active-config cutover failed because the deploy user could not write `rmhstudios-web-active.conf` (read-only filesystem) and `sudo` was blocked by `NoNewPrivileges`. Replace the privileged file rewrite with a mechanism the deploy user owns outright: a writable include directory, a reload over a socket, or a reverse-proxy upstream swap that needs no root.

Owner: Platform Eng **Target:** 4 Jul 2026 **Priority:** P0

B2 — Do not “add sudo support”

The previously proposed fix — granting the deploy sudo — is structurally wrong in a sandbox that sets `NoNewPrivileges` and is rejected. The cutover must be designed to require *no* privilege escalation at all.

Owner: Security Eng **Target:** 4 Jul 2026 **Priority:** P1

14.3 Defusing the trigger: a safe self-update

T1 — Re-derive environment before touching production

A self-updating deploy script must, upon restarting into its new version, fully re-derive its environment and configuration from the new commit *before* altering any production container — or refuse to manage `web` until it can prove the new configuration is in force. This eliminates the version-skew race of Chapter 6.

Owner: Lead SRE **Target:** 27 Jun 2026 **Priority:** P0

T2 — Make “except web” verifiable

Add an assertion that the orchestration configuration actually excludes `web` from the recreate step; fail the deploy loudly if the configuration and the control flow disagree, rather than silently tearing down `web`.

Owner: Platform Eng **Target:** 30 Jun 2026 **Priority:** P1

14.4 Compressing MTTR: detection and rollback

R1 — Synthetic uptime probe with paging

Add an external synthetic check that loads `RMHStudios.com` every 30s from outside the deployment host and pages on-call after two consecutive failures. The 21 June outage’s MTTR was dominated by detection-to-action latency (Chapter 13); this attacks that term directly.

Owner: SRE On-call **Target:** 28 Jun 2026 **Priority:** P0

R2 — One-command, tested rollback

Provide and rehearse a single command that restores the last-good container image. Rehearse it monthly in a game-day so that the legacy-fallback path used manually on 21 June becomes a first resort, not a discovery made hours in.

Owner: Lead SRE **Target:** 18 Jul 2026 **Priority:** P1

Table 14.1: Remediation items by fault, owner, and target date.

ID	Action	Owner	Target	Pri
A1	Gate teardown on health assertion	Lead SRE	27 Jun	P0
A2	Ordered startup, web first	Platform	30 Jun	P0
A3	Connection pooler (PgBouncer)	Database	11 Jul	P1
B1	Privilege-free cutover	Platform	4 Jul	P0
B2	Remove sudo dependency	Security	4 Jul	P1
T1	Safe self-update re-derivation	Lead SRE	27 Jun	P0
T2	Verifiable “except web”	Platform	30 Jun	P1
R1	Synthetic probe + paging	On-call	28 Jun	P0
R2	One-command tested rollback	Lead SRE	18 Jul	P1

14.5 Remediation summary

Key Finding

Five P0 items, all owned and dated within two weeks, remove the ability of any single deploy to both destroy blue and starve green. The remaining P1 items harden the system against recurrence and cut MTTR. No item depends on privilege escalation, and none re-introduces the structurally rejected “add sudo” approach.

CHAPTER 15

Lessons Learned

This chapter distils the incident into durable lessons — principles that outlive the specific bug and should shape how RMHSTUDIOS ships software. They are written to be blameless: every lesson points at a system property, not a person.

15.1 Lesson 1: Intentions are not invariants

The deploy step announced “all services except web” and believed it was protecting the live container. The configuration it drove tore that container down anyway. The gap between what the automation *intended* and what it *verified* is where the outage lived.

Key Finding

Every destructive action must be gated on a *checked assertion* about the world, never on the automation’s belief about what it is doing. “I am keeping blue live” must be a verified fact before blue is allowed to be destroyed — and if blue is being destroyed, that sentence is already false.

15.2 Lesson 2: Defence-in-depth requires independence

Six of seven protective layers failed together (Chapter 7) because they shared one hidden assumption: that blue would still exist when they ran. Layers stacked on a single foundation are not depth; they are one layer wearing seven coats.

15.3 Lesson 3: Simultaneity is a load-bearing decision

The connection storm (Chapter 9) was not a capacity shortfall. The same work, serialised or multiplexed, never approached saturation. “Recreate everything at once” is a performance decision disguised as a convenience, and on 21 June it decided the outage.

15.4 Lesson 4: Self-modifying automation needs a barrier

A script that rewrites and re-executes itself mid-run (Chapter 6) must treat the restart as a hard barrier: re-derive everything from the new commit, prove the new world is in force, *then* proceed. Version skew between new control flow and old configuration is a category of bug that only self-modifying automation can produce, and it must be designed out explicitly.

15.5 Lesson 5: MTTR is half of availability

The nines depend on $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$ (Chapter 13). Preventing incidents raises MTBF; recovering quickly lowers MTTR; both move the number. The 21 June MTTR of 347 minutes was dominated by detection-to-action latency — the cheapest minutes in the whole incident to win back.

15.6 Lesson 6: Timing turns downtime into lost trust

The same 660 minutes in a quiet week would have cost a fraction of what they cost immediately after the RMH Capital launch and RMH Space teaser (Chapters 11, 12). Release choreography should treat a flagship launch window as a change-freeze period: the worst time to ship a risky deploy is the moment the most new eyes are watching.

15.7 Lesson 7: Dismiss red herrings explicitly

The DeepSeek 401 `invalid api key` warnings during cover generation were alarming in the logs and causally irrelevant: the system fell back gracefully and the outage had a wholly separate origin. Naming and dismissing the red herring (Chapter 1) prevents the next responder from chasing it for an hour.

Table 15.1: Lessons mapped to the remediation items that institutionalise them.

#	Lesson	Items
1	Intentions are not invariants	A1, T2
2	Defence-in-depth needs independence	A1, A2, R2
3	Simultaneity is a decision	A2, A3
4	Self-modifying automation needs a barrier	T1, T2
5	MTTR is half of availability	R1, R2
6	Timing turns downtime into lost trust	Release policy
7	Dismiss red herrings explicitly	PIR practice

15.8 The unifying structure: an AND-gate that should have held

Every chapter of this review circles the same shape. The outage required *two* independent things to be true at once — blue removed *and* green unhealthy — and the whole purpose of a blue/green deployment is to guarantee those two can never be true together. The incident is therefore not a story about a single broken part; it is a story about a safety property that was assumed rather than enforced.

Why independence was the real failure

The fault tree of Chapter 7 gave the outage probability as a product,

$$PO = PB \cdot PG,$$

which is small *only if* the two events are independent. The version-skew trigger of Chapter 6 coupled them: the same mid-run `deploy.sh` self-update that tore down blue was part of the same control flow that mismanaged green. When a common cause drives both terms, the product collapses toward

a single probability,

$$p_O \rightarrow p_{\text{common}},$$

and defence-in-depth provides no defence at all. The mathematics of two nines multiplying into four only works when the two layers cannot fail for the same reason.

This reframes the entire remediation plan. Items A1–A3, T1–T2 and R1–R2 are not seven unrelated fixes; they are seven ways of restoring the independence the AND-gate silently lost — ordering teardown after confirmation (A1), staggering startup (A2–A3), barricading self-modifying automation (T1–T2), and shrinking the detection-to-action gap so a coupled failure is caught before it becomes an outage (R1–R2). Read together, they reconstruct the guarantee the deployment strategy was supposed to provide on its own.

Key Finding

The single sentence that captures this incident: *a deploy destroyed its own safety net because nothing forced it to check that the net was still there*. Every lesson above is a variation on making that check mandatory, independent, and fast.

APPENDIX A

Annotated Deploy Logs

This appendix reproduces sanitised excerpts of the four production deploy logs that constitute the evidentiary basis for this review. To keep the listings readable, long runs of identical lines (the repeated `curl` timeouts, `prisma:error` messages, Docker layer-pull progress, and library cover-generation entries) have been collapsed with explicit markers of the form `[... N further identical lines collapsed ...]`; the middle of each very long log is elided where marked. Box-drawing and other non-ASCII characters have been transliterated for typesetting. The original logs are retained unmodified in the incident record under their commit hashes.

Each excerpt should be read against the analysis chapters: 0a659e7b against Chapters 4–6, and the three afternoon deploys against Chapter 5.

A.1 Deploy 0a659e7b — 05:13 UTC (the outage deploy)

This is the morning deploy that caused the outage. Read for four things in sequence: (1) the self-update line `deploy.sh` was updated during pull -- restarting with the new version; (2) the library cover-generation block where the DeepSeek 401 red herring appears and is gracefully handled; (3) the Starting containers (all services except web) announcement *followed by the* `rmhstudios-prod-web` Stopping/Stopped/Removing/Removed sequence; and (4) the 60-second health-probe failure ending in green never became healthy -- keeping blue live, no traffic moved.

```
[2026-06-21 05:13:22] [production] Fetching latest code...
From github.com:stickms/rmhstudios.com
* branch          main          -> FETCH_HEAD
  739c32a6..0a659e7b  main      -> origin/main
M       data/library-metadata.json
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
HEAD is now at 0a659e7b feat(deploy): implement blue/green deployment strategy for web
container
[2026-06-21 05:13:25] [production]  \- done in 3s
[2026-06-21 05:13:25] [production] deploy.sh was updated during pull -- restarting with the
new version.
[2026-06-21 05:13:25] [production] === Deploy triggered (production, branch=main) ===
[2026-06-21 05:13:25] [production] Fetching latest code...
From github.com:stickms/rmhstudios.com
* branch          main          -> FETCH_HEAD
Your branch is up to date with 'origin/main'.
HEAD is now at 0a659e7b feat(deploy): implement blue/green deployment strategy for web
container
[2026-06-21 05:13:25] [production]  \- done in 0s
[2026-06-21 05:13:25] [production] Ensuring CDN asset permissions...
[2026-06-21 05:13:25] [production]  \- done in 0s
[2026-06-21 05:13:26] [production] Pre-build disk cleanup...
[2026-06-21 05:13:26] [production] Disk healthy: 15G free -- keeping build cache warm for a
fast incremental build.
[2026-06-21 05:13:26] [production]  \- done in 0s
[2026-06-21 05:13:26] [production] Generating library covers + metadata...
Found 24 PDF(s) in public/library.
* RMH_Capital_Board_Pitch_200p-1.pdf -- processing...
  ? DeepSeek failed for RMH_Capital_Board_Pitch_200p-1.pdf: 401 Authentication Fails, Your api
key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Board Pitch 200p 1" (200 pages) +cover
```



```
* RMH_Capital_Operating_Atlas_2026.pdf -- processing...
? DeepSeek failed for RMH_Capital_Operating_Atlas_2026.pdf: 401 Authentication Fails, Your
  api key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Operating Atlas 2026" (154 pages) +cover
* RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf -- processing...
-> "RMH Capital RMHCombinator 1000 Page Strategy CLEAN 2" (1000 pages) +cover
? DeepSeek failed for RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf: 401
  Authentication Fails, Your api key: ****bc3" is invalid. Using fallback.
* The_Reach_Equation.pdf -- processing...
? DeepSeek failed for The_Reach_Equation.pdf: 401 Authentication Fails, Your api key: ****
  bc3" is invalid. Using fallback.
-> "The Reach Equation" (83 pages) +cover
* The_Uncommitted_Self_100.pdf -- processing...
-> "The Uncommitted Self 100" (101 pages) +cover
? DeepSeek failed for The_Uncommitted_Self_100.pdf: 401 Authentication Fails, Your api key:
  ****bc3" is invalid. Using fallback.
* Wu_Thirteen_Minutes_Thesis.pdf -- cached
* elegy-of-the-mandate.pdf -- cached
* everything-platform-1000.pdf -- cached
* everything-platform-daily-bible.pdf -- cached
* everything-platform-holy-scriptures.pdf -- cached
* everything-platform-startup-book.pdf -- cached
* everything-platform.pdf -- cached
* everything_platform_build_plan-1.pdf -- cached
* everything_platform_build_plan.pdf -- cached
* everything_platform_minute_vol1.pdf -- cached
* everything_platform_minute_vol2.pdf -- cached
* everything_platform_minute_vol3.pdf -- cached
* everything_platform_minute_vol4.pdf -- cached
* everything_platform_minute_vol5.pdf -- cached
* everything_platform_minute_vol6.pdf -- cached
* kitab-al-alim-al-musaid.pdf -- cached
* rmhstudios-go-migration.pdf -- cached
* rmhtech-the-co-scientist-thesis-1.pdf -- cached
* rmhtech-the-co-scientist-thesis.pdf -- cached

Wrote 24 entries to data/library-metadata.json
[2026-06-21 05:13:52] [production]  '- done in 26s
[2026-06-21 05:13:52] [production] Building Docker image...
#0 building with "default" instance using docker driver

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#2 [web] resolve image config for docker-image://docker.io/docker/dockerfile:1
#2 DONE 0.1s

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#3 [web] docker-image://docker.io/docker/dockerfile:1@sha256:87999
  aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
#3 CACHED

#4 [web internal] load build definition from Dockerfile
#4 DONE 0.0s

#5 [web internal] load metadata for docker.io/library/node:24-alpine
#5 DONE 0.1s

#6 [web internal] load .dockerignore
#6 transferring context: 3.59kB done
#6 DONE 0.0s

#7 [web deps 1/5] FROM docker.io/library/node:24-alpine@sha256:156
  b55f92e98ccd5ef49578a8cea0df4679826564bad1c9d4ef04462b9f0ded6
#7 DONE 0.0s

#8 [web internal] load build context
#8 transferring context: 642.70kB 0.3s done
#8 DONE 0.3s
```

```
#9 [web deps 5/5] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/store,
  sharing=locked      pnpm install --frozen-lockfile --ignore-scripts
#9 CACHED

#10 [web prisma-generate 1/3] COPY prisma ./prisma/
#10 CACHED

#11 [web prisma-generate 2/3] COPY prisma.config.ts ./
#11 CACHED

#12 [web prisma-generate 3/3] RUN pnpm exec prisma generate
#12 CACHED

#13 [web deps 2/5] RUN corepack enable && corepack prepare pnpm@10.29.1 --activate
#13 CACHED

#14 [web deps 3/5] WORKDIR /app
#14 CACHED

#15 [web deps 4/5] COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
#15 CACHED

#16 [web vite-builder 1/6] COPY public ./public/
#16 CACHED

#17 [web vite-builder 2/6] COPY . .
#17 DONE 4.9s

#18 [web vite-builder 3/6] RUN pnpm run library:metadata
#18 0.626
#18 0.626 > rmhstudios.com@0.1.0 library:metadata /app
#18 0.626 > pnpm exec tsx scripts/generate-library-metadata.ts
  [... 4 further identical lines collapsed ...]

#19 [web vite-builder 4/6] RUN --mount=type=cache,id=vixi-cache-rmhstudios-prod,target=/app/.
  vixi,sharing=locked      rm -rf .output      && pnpm run build-vibe-packages      &&
  NODE_OPTIONS='--max-old-space-size=8192' pnpm exec vite build      && node scripts/fix-ssr-
  css-hash.mjs      && cp -a .output /app/build-output
#19 0.640
#19 0.640 > rmhstudios.com@0.1.0 build-vibe-packages /app
#19 0.640 > pnpm exec tsx scripts/build-vibe-packages.ts
  [... 24 further identical lines collapsed ...]
transforming...[ok] 8682 modules transformed.
#19 62.99 rendering chunks...
#19 64.14 .output/public/assets/KaTeX_Size3-Regular-CTq5MqoE.woff                4.42 kB
#19 64.14 .output/public/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2             4.92 kB
#19 64.14 .output/public/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2             5.20 kB
  [... 593 further identical lines collapsed ...]
transforming...[ok] 2102 modules transformed.
#19 73.12 rendering chunks...
#19 73.67 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size3-Regular-CTq5MqoE.woff
  4.42 kB
#19 73.67 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2
  4.92 kB
#19 73.67 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2
  5.20 kB
  [... 516 further identical lines collapsed ...]
transforming...[ok] 5743 modules transformed.
#19 76.49 ? Tracing dependencies:
#19 76.49 - @prisma/client (7.8.0)
#19 76.49 - esbuild (0.27.7)
#19 76.49 - satori (0.25.0)
  [... 586 further identical lines collapsed ...]

#20 [web vite-builder 5/6] RUN test -d /app/build-output &&      test -f /app/build-output/
  server/index.mjs &&      test -f /app/build-output/public/models/marlonjack.glb &&      test
  -f /app/build-output/public/vibe-packages/react.js
#20 DONE 0.2s

#21 [web vite-builder 6/6] RUN rm -rf /app/build-output/public/library      /app/build-
  output/public/models      /app/build-output/public/music      /app/build-
  output/public/sprites
#21 DONE 0.2s
```

```
#22 [web runner 2/13] RUN apk add --no-cache curl git chromium nss freetype harfbuzz ca-  
certificates ttf-freefont font-noto-emoji  
#22 CACHED  
  
#23 [web runner 3/13] WORKDIR /app  
#23 CACHED  
  
#24 [web prod-deps 1/1] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/  
store,sharing=locked pnpm install --frozen-lockfile --prod --ignore-scripts  
#24 CACHED  
  
#25 [web runner 4/13] RUN addgroup --system --gid 1001 nodejs && adduser --system --uid  
1001 app && mkdir -p /app/.rmhbot-worktrees && chown app:nodejs /app/.rmhbot-worktrees  
#25 CACHED  
  
#26 [web server-builder 13/17] COPY lib/rmhark-ai ./lib/rmhark-ai/  
#26 CACHED  
  
#27 [web server-builder 10/17] COPY lib/lights-out ./lib/lights-out/  
#27 CACHED  
  
#28 [web server-builder 8/17] COPY lib/baccarat ./lib/baccarat/  
#28 CACHED  
  
#29 [web server-builder 6/17] COPY lib/blackjack ./lib/blackjack/  
#29 CACHED  
  
#30 [web server-builder 16/17] RUN pnpm exec esbuild server/socket-server/index.ts  
server/rmhbox/index.ts server/rmhtube/index.ts server/recap/index.ts server/  
discord-bot/index.ts server/doctrine-worker/index.ts server/vibe-worker/index.ts  
server/bot-worker/index.ts --bundle --platform=node --target=node20 --outdir=  
dist-server --outbase=. --format=cjs --out-extension=.js=.cjs --packages=external --  
tree-shaking=true --tsconfig=tsconfig.server.json  
#30 CACHED  
  
#31 [web server-builder 7/17] COPY lib/holdem ./lib/holdem/  
#31 CACHED  
  
#32 [web server-builder 14/17] COPY lib/prisma.server.ts ./lib/prisma.server.ts  
#32 CACHED  
  
#33 [web server-builder 3/17] COPY lib/rmhbox ./lib/rmhbox/  
#33 CACHED  
  
#34 [web server-builder 15/17] COPY lib/url.ts ./lib/url.ts  
#34 CACHED  
  
#35 [web server-builder 12/17] COPY lib/rmhvibe ./lib/rmhvibe/  
#35 CACHED  
  
#36 [web server-builder 4/17] COPY lib/rmhtube ./lib/rmhtube/  
#36 CACHED  
  
#37 [web server-builder 1/17] COPY tsconfig.json tsconfig.server.json ./  
#37 CACHED  
  
#38 [web server-builder 9/17] COPY lib/roulette ./lib/roulette/  
#38 CACHED  
  
#39 [web server-builder 11/17] COPY lib/doctrine ./lib/doctrine/  
#39 CACHED  
  
#40 [web server-builder 5/17] COPY lib/rmhmusic ./lib/rmhmusic/  
#40 CACHED  
  
#41 [web server-builder 2/17] COPY server ./server/  
#41 CACHED  
  
#42 [web runner 5/13] COPY --from=prod-deps --chown=app:nodejs /app/node_modules ./  
node_modules  
#42 CACHED  
  
#43 [web server-builder 17/17] RUN test -f dist-server/server/socket-server/index.cjs &&  
test -f dist-server/server/rmhbox/index.cjs && test -f dist-server/server/rmhtube/
```

```
index.cjs &&      test -f dist-server/server/recap/index.cjs &&      test -f dist-server/
server/discord-bot/index.cjs &&      test -f dist-server/server/doctrine-worker/index.cjs
&&      test -f dist-server/server/vibe-worker/index.cjs &&      test -f dist-server/server/
bot-worker/index.cjs
#43 CACHED

#44 [web runner 6/13] COPY --from=vite-builder --chown=app:nodejs /app/build-output ./output
#44 DONE 1.6s

#45 [web runner 7/13] COPY --from=server-builder --chown=app:nodejs /app/dist-server ./dist-
server
#45 DONE 0.1s

#46 [web runner 8/13] COPY --chown=app:nodejs scripts ./scripts
#46 DONE 0.1s

#47 [web runner 9/13] COPY --chown=app:nodejs content ./content
#47 DONE 0.1s

#48 [web runner 10/13] COPY --chown=app:nodejs data ./data
#48 DONE 0.1s

#49 [web runner 11/13] COPY --chown=app:nodejs prisma ./prisma
#49 DONE 0.1s

#50 [web runner 12/13] COPY --chown=app:nodejs prisma.config.ts ./prisma.config.ts
#50 DONE 0.1s

#51 [web runner 13/13] COPY --chown=app:nodejs package.json ./package.json
#51 DONE 0.1s

#52 [web] exporting to image
#52 exporting layers
#52 exporting layers 0.5s done
#52 writing image sha256:ac64349a8a5198f4100aca407b64e80ed4876fc0660f24b50556edf90f404950 done
[... 2 further identical lines collapsed ...]

#53 [web] resolving provenance for metadata file
#53 DONE 0.0s
[2026-06-21 05:15:32] [production]  '- done in 100s
[2026-06-21 05:15:32] [production] Running database migrations...
[2026-06-21 05:15:35] [production] Database is reachable.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"

19 migrations found in prisma/migrations

No pending migrations to apply.
[2026-06-21 05:15:39] [production]  '- done in 7s
[2026-06-21 05:15:39] [production] Starting containers (all services except web)...
Container rmhstudios-prod-rmhtube Recreate
Container rmhstudios-prod-bot-worker Recreate
Container rmhstudios-prod-discord-bot Recreate
Container rmhstudios-prod-doctrine-worker Recreate
Container rmhstudios-prod-web Stopping
Container rmhstudios-prod-socket Recreate
Container rmhstudios-prod-rmhbox Recreate
Container rmhstudios-prod-vibe-worker Recreate
Container rmhstudios-prod-minio Running
Container rmhstudios-prod-recap Recreate
Container rmhstudios-prod-minio-init Creating
Container rmhstudios-prod-minio-init Created
Container rmhstudios-prod-discord-bot Recreated
Container rmhstudios-prod-rmhtube Recreated
Container rmhstudios-prod-doctrine-worker Recreated
Container rmhstudios-prod-recap Recreated
Container rmhstudios-prod-bot-worker Recreated
Container rmhstudios-prod-rmhbox Recreated
Container rmhstudios-prod-vibe-worker Recreated
```

```

Container rmhstudios-prod-socket Recreated
Container rmhstudios-prod-web Stopped
Container rmhstudios-prod-web Removing
Container rmhstudios-prod-web Removed
Container rmhstudios-prod-rmhtube Starting
Container rmhstudios-prod-vibe-worker Starting
Container rmhstudios-prod-socket Starting
Container rmhstudios-prod-discord-bot Starting
Container rmhstudios-prod-bot-worker Starting
Container rmhstudios-prod-doctrine-worker Starting
Container rmhstudios-prod-rmhbox Starting
Container rmhstudios-prod-recap Starting
Container rmhstudios-prod-minio Waiting
Container rmhstudios-prod-vibe-worker Started
Container rmhstudios-prod-minio Healthy
Container rmhstudios-prod-minio-init Starting
Container rmhstudios-prod-doctrine-worker Started
Container rmhstudios-prod-rmhtube Started
Container rmhstudios-prod-socket Started
Container rmhstudios-prod-bot-worker Started
Container rmhstudios-prod-discord-bot Started
Container rmhstudios-prod-recap Started
Container rmhstudios-prod-rmhbox Started
Container rmhstudios-prod-minio-init Started
[2026-06-21 05:15:48] [production]  `~ done in 9s
[2026-06-21 05:15:48] [production] Hotswapping web container (blue/green)...
[hotswap] Live port: 7005. Bringing up green on 7015 from rmhstudios-prod-app:latest.
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (52) Empty reply from server
curl: (28) Operation timed out after 3000 milliseconds with 0 bytes received
curl: (28) Operation timed out after 3001 milliseconds with 0 bytes received
curl: (28) Operation timed out after 3001 milliseconds with 0 bytes received
[... 56 further identical lines collapsed ...]
prisma:error Connection terminated due to connection timeout
prisma:error Connection terminated due to connection timeout
prisma:error Connection terminated due to connection timeout
prisma:error Connection terminated due to connection timeout
[... 46 further identical lines collapsed ...]
[hotswap] ERROR: rmhstudios-prod-web-green never became healthy -- keeping blue live, no
traffic moved.
[2026-06-21 05:19:48] [production] ERROR: web hotswap failed.

```

A.2 Deploy 0a6e5ddd — 11:41 UTC (first afternoon deploy)

The first afternoon deploy. Confirm the absence of any web teardown events (the new configuration is now fully in force, so “all services except web” finally means what it says) and that green becomes healthy. Then observe the Apache active-config cutover failing against the read-only filesystem and the blocked privilege escalation described in Chapter 5. No fresh customer-facing outage results, because blue is never removed.

```

[2026-06-21 11:41:19] [production] Fetching latest code...
From github.com:stickms/rmhstudios.com
* branch          main          -> FETCH_HEAD
   0a659e7b..0a6e5ddd  main       -> origin/main
M       data/library-metadata.json
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
HEAD is now at 0a6e5ddd refactor(hotswap): improve script portability and container management
[2026-06-21 11:41:22] [production]  `~ done in 3s
[2026-06-21 11:41:22] [production] Ensuring CDN asset permissions...
[2026-06-21 11:41:22] [production]  `~ done in 0s
[2026-06-21 11:41:23] [production] Pre-build disk cleanup...
Untagged: rmhstudios-prod-app:0b56a484
Deleted: sha256:9dfb86d2678dcb8de62dd7d4612eda60adfd6145fe9951bbe4ad4de9a97c52a4
[2026-06-21 11:41:23] [production] Disk healthy: 14G free -- keeping build cache warm for a
fast incremental build.
[2026-06-21 11:41:23] [production]  `~ done in 0s
[2026-06-21 11:41:23] [production] Generating library covers + metadata...
Found 24 PDF(s) in public/library.
* RMH_Capital_Board_Pitch_200p-1.pdf -- processing...

```

```

? DeepSeek failed for RMH_Capital_Board_Pitch_200p-1.pdf: 401 Authentication Fails, Your api
  key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Board Pitch 200p 1" (200 pages) +cover
* RMH_Capital_Operating_Atlas_2026.pdf -- processing...
? DeepSeek failed for RMH_Capital_Operating_Atlas_2026.pdf: 401 Authentication Fails, Your
  api key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Operating Atlas 2026" (154 pages) +cover
* RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf -- processing...
? DeepSeek failed for RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf: 401
  Authentication Fails, Your api key: ****bc3" is invalid. Using fallback.
-> "RMH Capital RMHCombinator 1000 Page Strategy CLEAN 2" (1000 pages) +cover
* The_Reach_Equation.pdf -- processing...
? DeepSeek failed for The_Reach_Equation.pdf: 401 Authentication Fails, Your api key: ****
  bc3" is invalid. Using fallback.
-> "The Reach Equation" (83 pages) +cover
* The_Uncommitted_Self_100.pdf -- processing...
-> "The Uncommitted Self 100" (101 pages) +cover
? DeepSeek failed for The_Uncommitted_Self_100.pdf: 401 Authentication Fails, Your api key:
  ****bc3" is invalid. Using fallback.
* Wu_Thirteen_Minutes_Thesis.pdf -- cached
* elegy-of-the-mandate.pdf -- cached
* everything-platform-1000.pdf -- cached
* everything-platform-daily-bible.pdf -- cached
* everything-platform-holy-scriptures.pdf -- cached
* everything-platform-startup-book.pdf -- cached
* everything-platform.pdf -- cached
* everything_platform_build_plan-1.pdf -- cached
* everything_platform_build_plan.pdf -- cached
* everything_platform_minute_vol1.pdf -- cached
* everything_platform_minute_vol2.pdf -- cached
* everything_platform_minute_vol3.pdf -- cached
* everything_platform_minute_vol4.pdf -- cached
* everything_platform_minute_vol5.pdf -- cached
* everything_platform_minute_vol6.pdf -- cached
* kitab-al-alim-al-musaid.pdf -- cached
* rmhstudios-go-migration.pdf -- cached
* rmhtech-the-co-scientist-thesis-1.pdf -- cached
* rmhtech-the-co-scientist-thesis.pdf -- cached

Wrote 24 entries to data/library-metadata.json
[2026-06-21 11:41:49] [production]   ^- done in 26s
[2026-06-21 11:41:49] [production] Building Docker image...
#0 building with "default" instance using docker driver

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#2 [web] resolve image config for docker-image://docker.io/docker/dockerfile:1
#2 DONE 0.1s

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#3 [web] docker-image://docker.io/docker/dockerfile:1@sha256:87999
  aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
#3 CACHED

#4 [web internal] load build definition from Dockerfile
#4 DONE 0.0s

#5 [web internal] load metadata for docker.io/library/node:24-alpine
#5 DONE 0.1s

#6 [web internal] load .dockerignore
#6 transferring context: 3.59kB done
#6 DONE 0.0s

#7 [web deps 1/5] FROM docker.io/library/node:24-alpine@sha256:156
  b55f92e98ccd5ef49578a8cea0df4679826564bad1c9d4ef04462b9f0ded6
#7 DONE 0.0s

#8 [web internal] load build context

```

```

#8 transferring context: 596.70kB 0.3s done
#8 DONE 0.3s

#9 [web deps 2/5] RUN corepack enable && corepack prepare pnpm@10.29.1 --activate
#9 CACHED

#10 [web deps 3/5] WORKDIR /app
#10 CACHED

#11 [web deps 4/5] COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
#11 CACHED

#12 [web deps 5/5] RUN --mount=type=cache,id=ppnm-store,target=/root/.local/share/pnpm/store,
    sharing=locked      pnpm install --frozen-lockfile --ignore-scripts
#12 CACHED

#13 [web prisma-generate 1/3] COPY prisma ./prisma/
#13 CACHED

#14 [web prisma-generate 2/3] COPY prisma.config.ts ./
#14 CACHED

#15 [web prisma-generate 3/3] RUN pnpm exec prisma generate
#15 CACHED

#16 [web vite-builder 1/6] COPY public ./public/
#16 CACHED

#17 [web vite-builder 2/6] COPY . .
#17 DONE 4.8s

#18 [web vite-builder 3/6] RUN pnpm run library:metadata
#18 0.642
#18 0.642 > rmhstudios.com@0.1.0 library:metadata /app
#18 0.642 > pnpm exec tsx scripts/generate-library-metadata.ts
    [... 4 further identical lines collapsed ...]

#19 [web vite-builder 4/6] RUN --mount=type=cache,id=vixxi-cache-rmhstudios-prod,target=/app/.
    vixxi,sharing=locked      rm -rf .output      && pnpm run build-vibe-packages      &&
    NODE_OPTIONS='--max-old-space-size=8192' pnpm exec vite build      && node scripts/fix-ssr-
    css-hash.mjs      && cp -a .output /app/build-output
#19 0.605
#19 0.605 > rmhstudios.com@0.1.0 build-vibe-packages /app
#19 0.605 > pnpm exec tsx scripts/build-vibe-packages.ts
    [... 24 further identical lines collapsed ...]
transforming...[ok] 8682 modules transformed.
#19 62.52 rendering chunks...
#19 63.70 .output/public/assets/KaTeX_Size3-Regular-CTq5MqoE.woff                4.42 kB
#19 63.70 .output/public/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2            4.92 kB
#19 63.70 .output/public/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2            5.20 kB
    [... 593 further identical lines collapsed ...]
transforming...[ok] 2102 modules transformed.
#19 72.70 rendering chunks...
#19 73.23 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size3-Regular-CTq5MqoE.woff
    4.42 kB
#19 73.23 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2
    4.92 kB
#19 73.23 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2
    5.20 kB
    [... 516 further identical lines collapsed ...]
transforming...[ok] 5743 modules transformed.
#19 76.06 ? Tracing dependencies:
#19 76.06 - @prisma/client (7.8.0)
#19 76.06 - esbuild (0.27.7)
#19 76.06 - @resvg/resvg-js (2.6.2)
    [... 586 further identical lines collapsed ...]

#20 [web vite-builder 5/6] RUN test -d /app/build-output &&      test -f /app/build-output/
    server/index.mjs &&      test -f /app/build-output/public/models/marlonjack.glb &&      test
    -f /app/build-output/public/vibe-packages/react.js
#20 DONE 0.2s

#21 [web vite-builder 6/6] RUN rm -rf /app/build-output/public/library      /app/build-
    output/public/models      /app/build-output/public/music      /app/build-

```

```
output/public/sprites
#21 DONE 0.2s

#22 [web runner 4/13] RUN addgroup --system --gid 1001 nodejs && adduser --system --uid
1001 app && mkdir -p /app/.rmhbot-worktrees && chown app:nodejs /app/.rmhbot-worktrees
#22 CACHED

#23 [web runner 3/13] WORKDIR /app
#23 CACHED

#24 [web prod-deps 1/1] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/
store,sharing=locked pnpm install --frozen-lockfile --prod --ignore-scripts
#24 CACHED

#25 [web runner 2/13] RUN apk add --no-cache curl git chromium nss freetype harfbuzz ca-
certificates ttf-freefont font-noto-emoji
#25 CACHED

#26 [web server-builder 13/17] COPY lib/rmhark-ai ./lib/rmhark-ai/
#26 CACHED

#27 [web server-builder 16/17] RUN pnpm exec esbuild server/socket-server/index.ts
server/rmhbox/index.ts server/rmhtube/index.ts server/recap/index.ts server/
discord-bot/index.ts server/doctrine-worker/index.ts server/vibe-worker/index.ts
server/bot-worker/index.ts --bundle --platform=node --target=node20 --outdir=
dist-server --outbase=. --format=cjs --out-extension=.js=.cjs --packages=external --
tree-shaking=true --tsconfig=tsconfig.server.json
#27 CACHED

#28 [web server-builder 8/17] COPY lib/baccarat ./lib/baccarat/
#28 CACHED

#29 [web server-builder 15/17] COPY lib/url.ts ./lib/url.ts
#29 CACHED

#30 [web server-builder 5/17] COPY lib/rmhmusic ./lib/rmhmusic/
#30 CACHED

#31 [web server-builder 6/17] COPY lib/blackjack ./lib/blackjack/
#31 CACHED

#32 [web server-builder 9/17] COPY lib/roulette ./lib/roulette/
#32 CACHED

#33 [web server-builder 12/17] COPY lib/rmhvibe ./lib/rmhvibe/
#33 CACHED

#34 [web server-builder 3/17] COPY lib/rmhbox ./lib/rmhbox/
#34 CACHED

#35 [web server-builder 10/17] COPY lib/lights-out ./lib/lights-out/
#35 CACHED

#36 [web server-builder 1/17] COPY tsconfig.json tsconfig.server.json ./
#36 CACHED

#37 [web server-builder 2/17] COPY server ./server/
#37 CACHED

#38 [web server-builder 4/17] COPY lib/rmhtube ./lib/rmhtube/
#38 CACHED

#39 [web server-builder 14/17] COPY lib/prisma.server.ts ./lib/prisma.server.ts
#39 CACHED

#40 [web server-builder 11/17] COPY lib/doctrine ./lib/doctrine/
#40 CACHED

#41 [web server-builder 7/17] COPY lib/holdem ./lib/holdem/
#41 CACHED

#42 [web runner 5/13] COPY --from=prod-deps --chown=app:nodejs /app/node_modules ./
node_modules
#42 CACHED
```



```
#43 [web server-builder 17/17] RUN test -f dist-server/server/socket-server/index.cjs &&
test -f dist-server/server/rmhbox/index.cjs && test -f dist-server/server/rmhtube/
index.cjs && test -f dist-server/server/recap/index.cjs && test -f dist-server/
server/discord-bot/index.cjs && test -f dist-server/server/doctrine-worker/index.cjs
&& test -f dist-server/server/vibe-worker/index.cjs && test -f dist-server/server/
bot-worker/index.cjs
#43 CACHED

#44 [web runner 6/13] COPY --from=vite-builder --chown=app:nodejs /app/build-output ./output
#44 DONE 1.6s

#45 [web runner 7/13] COPY --from=server-builder --chown=app:nodejs /app/dist-server ./dist-
server
#45 DONE 0.1s

#46 [web runner 8/13] COPY --chown=app:nodejs scripts ./scripts
#46 DONE 0.1s

#47 [web runner 9/13] COPY --chown=app:nodejs content ./content
#47 DONE 0.1s

#48 [web runner 10/13] COPY --chown=app:nodejs data ./data
#48 DONE 0.1s

#49 [web runner 11/13] COPY --chown=app:nodejs prisma ./prisma
#49 DONE 0.1s

#50 [web runner 12/13] COPY --chown=app:nodejs prisma.config.ts ./prisma.config.ts
#50 DONE 0.1s

#51 [web runner 13/13] COPY --chown=app:nodejs package.json ./package.json
#51 DONE 0.1s

#52 [web] exporting to image
#52 exporting layers
#52 exporting layers 0.5s done
#52 writing image sha256:016779d5f1c8f91dd9ddd49e078217ae718ee74d863e64d569fb8dc6c034c4d2 done
[... 2 further identical lines collapsed ...]

#53 [web] resolving provenance for metadata file
#53 DONE 0.0s
[2026-06-21 11:43:29] [production] `~ done in 100s
[2026-06-21 11:43:29] [production] Running database migrations...
[2026-06-21 11:43:31] [production] Database is reachable.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"

19 migrations found in prisma/migrations

No pending migrations to apply.
[2026-06-21 11:43:36] [production] `~ done in 7s
[2026-06-21 11:43:36] [production] Starting containers (all services except web)...
Container rmhstudios-prod-minio Running
Container rmhstudios-prod-minio-init Creating
Container rmhstudios-prod-doctrine-worker Recreate
Container rmhstudios-prod-vibe-worker Recreate
Container rmhstudios-prod-rmhtube Recreate
Container rmhstudios-prod-bot-worker Recreate
Container rmhstudios-prod-discord-bot Recreate
Container rmhstudios-prod-rmhbox Recreate
Container rmhstudios-prod-socket Recreate
Container rmhstudios-prod-recap Recreate
Container rmhstudios-prod-minio-init Created
Container rmhstudios-prod-doctrine-worker Recreated
Container rmhstudios-prod-discord-bot Recreated
Container rmhstudios-prod-recap Recreated
Container rmhstudios-prod-socket Recreated
Container rmhstudios-prod-bot-worker Recreated
```

```

Container rmhstudios-prod-vibe-worker Recreated
Container rmhstudios-prod-rmhtube Recreated
Container rmhstudios-prod-rmhbox Recreated
Container rmhstudios-prod-bot-worker Starting
Container rmhstudios-prod-doctrine-worker Starting
Container rmhstudios-prod-vibe-worker Starting
Container rmhstudios-prod-minio Waiting
Container rmhstudios-prod-socket Starting
Container rmhstudios-prod-discord-bot Starting
Container rmhstudios-prod-recap Starting
Container rmhstudios-prod-rmhtube Starting
Container rmhstudios-prod-rmhbox Starting
Container rmhstudios-prod-doctrine-worker Started
Container rmhstudios-prod-bot-worker Started
Container rmhstudios-prod-minio Healthy
Container rmhstudios-prod-minio-init Starting
Container rmhstudios-prod-vibe-worker Started
Container rmhstudios-prod-discord-bot Started
Container rmhstudios-prod-rmhtube Started
Container rmhstudios-prod-socket Started
Container rmhstudios-prod-rmhbox Started
Container rmhstudios-prod-recap Started
Container rmhstudios-prod-minio-init Started
[2026-06-21 11:43:38] [production] 	'- done in 2s
[2026-06-21 11:43:38] [production] Hotswapping web container (blue/green)...
[hotswap] Live port: 7005. Bringing up green on 7015 from rmhstudios-prod-app:latest.
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (52) Empty reply from server
[hotswap] rmhstudios-prod-web-green is healthy.
/home/rmhstudios/rmhstudios.com/deploy/hotswap-web.sh: line 124: /etc/apache2/conf-available/
rmhstudios-web-active.conf: Read-only file system
[2026-06-21 11:43:41] [production] ERROR: web hotswap failed.

```

A.3 Deploy 2e46b0cd — 14:45 UTC (second afternoon deploy)

A near-repeat of the first afternoon deploy. The same cutover wall recurs; the same protective absence of a web teardown keeps the site up throughout.

```

[2026-06-21 14:45:50] [production] Fetching latest code...
From github.com:stickms/rmhstudios.com
* branch          main          -> FETCH_HEAD
0a6e5ddd..2e46b0cd main      -> origin/main
M   data/library-metadata.json
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
HEAD is now at 2e46b0cd fix(hotswap): add sudo support for Apache configuration updates
[2026-06-21 14:45:53] [production] 	'- done in 3s
[2026-06-21 14:45:53] [production] Ensuring CDN asset permissions...
[2026-06-21 14:45:53] [production] 	'- done in 0s
[2026-06-21 14:45:54] [production] Pre-build disk cleanup...
Untagged: rmhstudios-prod-app:739c32a6
Deleted: sha256:ccb43c52ffeld52a68eeldea27b3bd50e1c6d92597fd5eble920975eaa5b796
[2026-06-21 14:45:54] [production] Disk healthy: 13G free -- keeping build cache warm for a
fast incremental build.
[2026-06-21 14:45:54] [production] 	'- done in 0s
[2026-06-21 14:45:54] [production] Generating library covers + metadata...
Found 24 PDF(s) in public/library.
* RMH_Capital_Board_Pitch_200p-1.pdf -- processing...
? DeepSeek failed for RMH_Capital_Board_Pitch_200p-1.pdf: 401 Authentication Fails, Your api
key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Board Pitch 200p 1" (200 pages) +cover
* RMH_Capital_Operating_Atlas_2026.pdf -- processing...
-> "RMH Capital Operating Atlas 2026" (154 pages) +cover
? DeepSeek failed for RMH_Capital_Operating_Atlas_2026.pdf: 401 Authentication Fails, Your
api key: ****bc3" is invalid. Using fallback.
* RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf -- processing...
? DeepSeek failed for RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf: 401
Authentication Fails, Your api key: ****bc3" is invalid. Using fallback.
-> "RMH Capital RMHCombinator 1000 Page Strategy CLEAN 2" (1000 pages) +cover
* The_Reach_Equation.pdf -- processing...

```

```
? DeepSeek failed for The_Reach_Equation.pdf: 401 Authentication Fails, Your api key: ****
bc3" is invalid. Using fallback.
-> "The Reach Equation" (83 pages) +cover
* The_Uncommitted_Self_100.pdf -- processing...
? DeepSeek failed for The_Uncommitted_Self_100.pdf: 401 Authentication Fails, Your api key:
****bc3" is invalid. Using fallback.
-> "The Uncommitted Self 100" (101 pages) +cover
* Wu Thirteen Minutes Thesis.pdf -- cached
* elegy-of-the-mandate.pdf -- cached
* everything-platform-1000.pdf -- cached
* everything-platform-daily-bible.pdf -- cached
* everything-platform-holy-scriptures.pdf -- cached
* everything-platform-startup-book.pdf -- cached
* everything-platform.pdf -- cached
* everything_platform_build_plan-1.pdf -- cached
* everything_platform_build_plan.pdf -- cached
* everything_platform_minute_vol1.pdf -- cached
* everything_platform_minute_vol2.pdf -- cached
* everything_platform_minute_vol3.pdf -- cached
* everything_platform_minute_vol4.pdf -- cached
* everything_platform_minute_vol5.pdf -- cached
* everything_platform_minute_vol6.pdf -- cached
* kitab-al-alim-al-musaid.pdf -- cached
* rmhstudios-go-migration.pdf -- cached
* rmhtech-the-co-scientist-thesis-1.pdf -- cached
* rmhtech-the-co-scientist-thesis.pdf -- cached

Wrote 24 entries to data/library-metadata.json
[2026-06-21 14:46:20] [production]  `~ done in 26s
[2026-06-21 14:46:20] [production] Building Docker image...
#0 building with "default" instance using docker driver

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#2 [web] resolve image config for docker-image://docker.io/docker/dockerfile:1
#2 DONE 0.1s

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#3 [web] docker-image://docker.io/docker/dockerfile:1@sha256:87999
aa3d42bdc6bea60565083ee17e86dlf3339802f543c0d03998580f9cb89
#3 CACHED

#4 [web internal] load build definition from Dockerfile
#4 DONE 0.0s

#5 [web internal] load metadata for docker.io/library/node:24-alpine
#5 DONE 0.1s

#6 [web internal] load .dockerignore
#6 transferring context: 3.59kB done
#6 DONE 0.0s

#7 [web deps 1/5] FROM docker.io/library/node:24-alpine@sha256:156
b55f92e98ccd5ef49578a8cea0df4679826564bad1c9d4ef04462b9f0ded6
#7 DONE 0.0s

#8 [web internal] load build context
#8 transferring context: 596.73kB 0.3s done
#8 DONE 0.3s

#9 [web deps 4/5] COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
#9 CACHED

#10 [web deps 5/5] RUN --mount=type=cache,id=ppnpm-store,target=/root/.local/share/ppnpm/store,
sharing=locked      pnpm install --frozen-lockfile --ignore-scripts
#10 CACHED

#11 [web prisma-generate 1/3] COPY prisma ./prisma/
#11 CACHED
```

```
#12 [web prisma-generate 2/3] COPY prisma.config.ts ./
#12 CACHED

#13 [web deps 2/5] RUN corepack enable && corepack prepare pnpm@10.29.1 --activate
#13 CACHED

#14 [web deps 3/5] WORKDIR /app
#14 CACHED

#15 [web prisma-generate 3/3] RUN pnpm exec prisma generate
#15 CACHED

#16 [web vite-builder 1/6] COPY public ./public/
#16 CACHED

#17 [web vite-builder 2/6] COPY . .
#17 DONE 5.2s

#18 [web vite-builder 3/6] RUN pnpm run library:metadata
#18 0.660
#18 0.660 > rmhstudios.com@0.1.0 library:metadata /app
#18 0.660 > pnpm exec tsx scripts/generate-library-metadata.ts
[... 4 further identical lines collapsed ...]

#19 [web vite-builder 4/6] RUN --mount=type=cache,id=vinxi-cache-rmhstudios-prod,target=/app/.
  vinxi,sharing=locked    rm -rf .output    && pnpm run build-vibe-packages    &&
  NODE_OPTIONS='--max-old-space-size=8192' pnpm exec vite build    && node scripts/fix-ssr-
  css-hash.mjs    && cp -a .output /app/build-output
#19 0.577
#19 0.577 > rmhstudios.com@0.1.0 build-vibe-packages /app
#19 0.577 > pnpm exec tsx scripts/build-vibe-packages.ts
[... 24 further identical lines collapsed ...]
transforming...[ok] 8682 modules transformed.
#19 63.27 rendering chunks...
#19 64.51 .output/public/assets/KaTeX_Size3-Regular-CTq5MqoE.woff                4.42 kB
#19 64.51 .output/public/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2             4.92 kB
#19 64.51 .output/public/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2             5.20 kB
[... 593 further identical lines collapsed ...]
transforming...[ok] 2102 modules transformed.
#19 73.42 rendering chunks...
#19 73.98 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size3-Regular-CTq5MqoE.woff
  4.42 kB
#19 73.98 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2
  4.92 kB
#19 73.98 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2
  5.20 kB
[... 516 further identical lines collapsed ...]
transforming...[ok] 5743 modules transformed.
#19 76.83 ? Tracing dependencies:
#19 76.83 - esbuild (0.27.7)
#19 76.83 - @prisma/client (7.8.0)
#19 76.83 - @resvg/resvg-js (2.6.2)
[... 586 further identical lines collapsed ...]

#20 [web vite-builder 5/6] RUN test -d /app/build-output &&    test -f /app/build-output/
  server/index.mjs &&    test -f /app/build-output/public/models/marlonjack.glb &&    test
  -f /app/build-output/public/vibe-packages/react.js
#20 DONE 0.2s

#21 [web vite-builder 6/6] RUN rm -rf /app/build-output/public/library    /app/build-
  output/public/models    /app/build-output/public/music    /app/build-
  output/public/sprites
#21 DONE 0.2s

#22 [web prod-deps 1/1] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/
  store,sharing=locked    pnpm install --frozen-lockfile --prod --ignore-scripts
#22 CACHED

#23 [web runner 3/13] WORKDIR /app
#23 CACHED

#24 [web runner 2/13] RUN apk add --no-cache curl git    chromium nss freetype harfbuzz ca-
  certificates ttf-freefont font-noto-emoji
```

```
#24 CACHED

#25 [web runner 4/13] RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 app && mkdir -p /app/.rmhbot-worktrees && chown app:nodejs /app/.rmhbot-worktrees
#25 CACHED

#26 [web server-builder 16/17] RUN pnpm exec esbuild server/socket-server/index.ts server/rmhbox/index.ts server/rmhtube/index.ts server/recap/index.ts server/discord-bot/index.ts server/doctrine-worker/index.ts server/vibe-worker/index.ts server/bot-worker/index.ts --bundle --platform=node --target=node20 --outdir=dist-server --outbase=. --format=cjs --out-extension:.js=.cjs --packages=external --tree-shaking=true --tsconfig=tsconfig.server.json
#26 CACHED

#27 [web server-builder 6/17] COPY lib/blackjack ./lib/blackjack/
#27 CACHED

#28 [web server-builder 4/17] COPY lib/rmhtube ./lib/rmhtube/
#28 CACHED

#29 [web server-builder 8/17] COPY lib/baccarat ./lib/baccarat/
#29 CACHED

#30 [web server-builder 11/17] COPY lib/doctrine ./lib/doctrine/
#30 CACHED

#31 [web server-builder 14/17] COPY lib/prisma.server.ts ./lib/prisma.server.ts
#31 CACHED

#32 [web server-builder 15/17] COPY lib/url.ts ./lib/url.ts
#32 CACHED

#33 [web server-builder 2/17] COPY server ./server/
#33 CACHED

#34 [web server-builder 10/17] COPY lib/lights-out ./lib/lights-out/
#34 CACHED

#35 [web server-builder 9/17] COPY lib/roulette ./lib/roulette/
#35 CACHED

#36 [web server-builder 5/17] COPY lib/rmhmusic ./lib/rmhmusic/
#36 CACHED

#37 [web server-builder 3/17] COPY lib/rmhbox ./lib/rmhbox/
#37 CACHED

#38 [web server-builder 7/17] COPY lib/holdem ./lib/holdem/
#38 CACHED

#39 [web server-builder 13/17] COPY lib/rmhark-ai ./lib/rmhark-ai/
#39 CACHED

#40 [web server-builder 12/17] COPY lib/rmhvibe ./lib/rmhvibe/
#40 CACHED

#41 [web server-builder 1/17] COPY tsconfig.json tsconfig.server.json ./
#41 CACHED

#42 [web server-builder 17/17] RUN test -f dist-server/server/socket-server/index.cjs && test -f dist-server/server/rmhbox/index.cjs && test -f dist-server/server/rmhtube/index.cjs && test -f dist-server/server/recap/index.cjs && test -f dist-server/server/discord-bot/index.cjs && test -f dist-server/server/doctrine-worker/index.cjs && test -f dist-server/server/vibe-worker/index.cjs && test -f dist-server/server/bot-worker/index.cjs
#42 CACHED

#43 [web runner 5/13] COPY --from=prod-deps --chown=app:nodejs /app/node_modules ./node_modules
#43 CACHED

#44 [web runner 6/13] COPY --from=vite-builder --chown=app:nodejs /app/build-output ./output
#44 DONE 1.7s
```

```
#45 [web runner 7/13] COPY --from=server-builder --chown=app:nodejs /app/dist-server ./dist-
server
#45 DONE 0.1s

#46 [web runner 8/13] COPY --chown=app:nodejs scripts ./scripts
#46 DONE 0.1s

#47 [web runner 9/13] COPY --chown=app:nodejs content ./content
#47 DONE 0.1s

#48 [web runner 10/13] COPY --chown=app:nodejs data ./data
#48 DONE 0.1s

#49 [web runner 11/13] COPY --chown=app:nodejs prisma ./prisma
#49 DONE 0.1s

#50 [web runner 12/13] COPY --chown=app:nodejs prisma.config.ts ./prisma.config.ts
#50 DONE 0.1s

#51 [web runner 13/13] COPY --chown=app:nodejs package.json ./package.json
#51 DONE 0.1s

#52 [web] exporting to image
#52 exporting layers
#52 exporting layers 0.5s done
#52 writing image sha256:96f79eeefd191022513e3abda9e7425f23ac7ddale524a67c37219982062e065 done
[... 2 further identical lines collapsed ...]

#53 [web] resolving provenance for metadata file
#53 DONE 0.0s
[2026-06-21 14:48:01] [production] '- done in 101s
[2026-06-21 14:48:01] [production] Running database migrations...
[2026-06-21 14:48:04] [production] Database is reachable.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"

19 migrations found in prisma/migrations

No pending migrations to apply.
[2026-06-21 14:48:08] [production] '- done in 7s
[2026-06-21 14:48:08] [production] Starting containers (all services except web)...
Container rmhstudios-prod-minio Running
Container rmhstudios-prod-bot-worker Recreate
Container rmhstudios-prod-minio-init Creating
Container rmhstudios-prod-recap Recreate
Container rmhstudios-prod-rmhtube Recreate
Container rmhstudios-prod-doctrine-worker Recreate
Container rmhstudios-prod-socket Recreate
Container rmhstudios-prod-discord-bot Recreate
Container rmhstudios-prod-vibe-worker Recreate
Container rmhstudios-prod-rmhbox Recreate
Container rmhstudios-prod-minio-init Created
Container rmhstudios-prod-recap Recreated
Container rmhstudios-prod-bot-worker Recreated
Container rmhstudios-prod-vibe-worker Recreated
Container rmhstudios-prod-doctrine-worker Recreated
Container rmhstudios-prod-discord-bot Recreated
Container rmhstudios-prod-socket Recreated
Container rmhstudios-prod-rmhtube Recreated
Container rmhstudios-prod-rmhbox Recreated
Container rmhstudios-prod-rmhbox Starting
Container rmhstudios-prod-bot-worker Starting
Container rmhstudios-prod-recap Starting
Container rmhstudios-prod-discord-bot Starting
Container rmhstudios-prod-minio Waiting
Container rmhstudios-prod-vibe-worker Starting
Container rmhstudios-prod-doctrine-worker Starting
Container rmhstudios-prod-rmhtube Starting
Container rmhstudios-prod-socket Starting
```

```

Container rmhstudios-prod-rmhbox Started
Container rmhstudios-prod-minio Healthy
Container rmhstudios-prod-minio-init Starting
Container rmhstudios-prod-discord-bot Started
Container rmhstudios-prod-recap Started
Container rmhstudios-prod-socket Started
Container rmhstudios-prod-doctrine-worker Started
Container rmhstudios-prod-vibe-worker Started
Container rmhstudios-prod-rmhtube Started
Container rmhstudios-prod-bot-worker Started
Container rmhstudios-prod-minio-init Started
[2026-06-21 14:48:10] [production] `-- done in 2s
[2026-06-21 14:48:10] [production] Hotswapping web container (blue/green)...
[hotswap] Live port: 7005. Bringing up green on 7015 from rmhstudios-prod-app:latest.
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (52) Empty reply from server
[hotswap] rmhstudios-prod-web-green is healthy.
/home/rmhstudios/rmhstudios.com/deploy/hotswap-web.sh: line 64: /etc/apache2/conf-available/
rmhstudios-web-active.conf: Read-only file system
sudo: The "no new privileges" flag is set, which prevents sudo from running as root.
sudo: If sudo is running in a container, you may need to adjust the container configuration to
disable the flag.
[hotswap] ERROR: cannot write /etc/apache2/conf-available/rmhstudios-web-active.conf -- chown
it to the deploy user or grant sudo (see setup notes).
[2026-06-21 14:48:14] [production] ERROR: web hotswap failed.

```

A.4 Deploy 02493f03 — 15:02 UTC (third afternoon deploy)

The third afternoon deploy, including the structurally-incorrect “add sudo support” attempt that the sandbox’s NoNewPrivileges flag rejects — the fix that cannot work in this environment and which Chapter 14 replaces with a privilege-free cutover.

```

[2026-06-21 15:02:45] [production] Fetching latest code...
From github.com:stickms/rmhstudios.com
* branch          main          -> FETCH_HEAD
2e46b0cd..02493f03 main        -> origin/main
M   data/library-metadata.json
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
HEAD is now at 02493f03 fix(hotswap): enhance error logging and diagnostics for Apache
configuration updates
[2026-06-21 15:02:48] [production] `-- done in 3s
[2026-06-21 15:02:48] [production] Ensuring CDN asset permissions...
[2026-06-21 15:02:48] [production] `-- done in 0s
[2026-06-21 15:02:49] [production] Pre-build disk cleanup...
Untagged: rmhstudios-prod-app:0a659e7b
Deleted: sha256:ac64349a8a5198f4100aca07b64e80ed4876fc0660f24b50556edf90f404950
[2026-06-21 15:02:49] [production] Disk healthy: 13G free -- keeping build cache warm for a
fast incremental build.
[2026-06-21 15:02:49] [production] `-- done in 0s
[2026-06-21 15:02:49] [production] Generating library covers + metadata...
Found 24 PDF(s) in public/library.
* RMH_Capital_Board_Pitch_200p-1.pdf -- processing...
? DeepSeek failed for RMH_Capital_Board_Pitch_200p-1.pdf: 401 Authentication Fails, Your api
key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Board Pitch 200p 1" (200 pages) +cover
* RMH_Capital_Operating_Atlas_2026.pdf -- processing...
? DeepSeek failed for RMH_Capital_Operating_Atlas_2026.pdf: 401 Authentication Fails, Your
api key: ****bc3" is invalid. Using fallback.
-> "RMH Capital Operating Atlas 2026" (154 pages) +cover
* RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf -- processing...
-> "RMH Capital RMHCombinator 1000 Page Strategy CLEAN 2" (1000 pages) +cover
? DeepSeek failed for RMH_Capital_RMHCombinator_1000_Page_Strategy_CLEAN-2.pdf: 401
Authentication Fails, Your api key: ****bc3" is invalid. Using fallback.
* The_Reach_Equation.pdf -- processing...
-> "The Reach Equation" (83 pages) +cover
? DeepSeek failed for The_Reach_Equation.pdf: 401 Authentication Fails, Your api key: ****
bc3" is invalid. Using fallback.
* The_Uncommitted_Self_100.pdf -- processing...

```

```
? DeepSeek failed for The_Uncommitted_Self_100.pdf: 401 Authentication Fails, Your api key:
***bc3" is invalid. Using fallback.
-> "The Uncommitted Self 100" (101 pages) +cover
* Wu Thirteen Minutes Thesis.pdf -- cached
* elegy-of-the-mandate.pdf -- cached
* everything-platform-1000.pdf -- cached
* everything-platform-daily-bible.pdf -- cached
* everything-platform-holy-scriptures.pdf -- cached
* everything-platform-startup-book.pdf -- cached
* everything-platform.pdf -- cached
* everything_platform_build_plan-1.pdf -- cached
* everything_platform_build_plan.pdf -- cached
* everything_platform_minute_vol1.pdf -- cached
* everything_platform_minute_vol2.pdf -- cached
* everything_platform_minute_vol3.pdf -- cached
* everything_platform_minute_vol4.pdf -- cached
* everything_platform_minute_vol5.pdf -- cached
* everything_platform_minute_vol6.pdf -- cached
* kitab-al-alim-al-musaid.pdf -- cached
* rmhstudios-go-migration.pdf -- cached
* rmhtech-the-co-scientist-thesis-1.pdf -- cached
* rmhtech-the-co-scientist-thesis.pdf -- cached

Wrote 24 entries to data/library-metadata.json
[2026-06-21 15:03:15] [production]  `~ done in 26s
[2026-06-21 15:03:15] [production] Building Docker image...
#0 building with "default" instance using docker driver

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#2 [web] resolve image config for docker-image://docker.io/docker/dockerfile:1
#2 DONE 0.1s

#1 [web internal] load build definition from Dockerfile
#1 transferring dockerfile: 11.77kB done
#1 DONE 0.0s

#3 [web] docker-image://docker.io/docker/dockerfile:1@sha256:87999
aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
#3 CACHED

#4 [web internal] load build definition from Dockerfile
#4 DONE 0.0s

#5 [web internal] load metadata for docker.io/library/node:24-alpine
#5 DONE 0.1s

#6 [web internal] load .dockerignore
#6 transferring context: 3.59kB done
#6 DONE 0.0s

#7 [web deps 1/5] FROM docker.io/library/node:24-alpine@sha256:156
b55f92e98ccd5ef49578a8cea0df4679826564bad1c9d4ef04462b9f0ded6
#7 DONE 0.0s

#8 [web internal] load build context
#8 transferring context: 608.36kB 0.3s done
#8 DONE 0.3s

#9 [web prisma-generate 1/3] COPY prisma ./prisma/
#9 CACHED

#10 [web prisma-generate 2/3] COPY prisma.config.ts ./
#10 CACHED

#11 [web prisma-generate 3/3] RUN pnpm exec prisma generate
#11 CACHED

#12 [web deps 2/5] RUN corepack enable && corepack prepare pnpm@10.29.1 --activate
#12 CACHED

#13 [web deps 3/5] WORKDIR /app
```



```
#13 CACHED

#14 [web deps 4/5] COPY package.json pnpm-lock.yaml pnpm-workspace.yaml ./
#14 CACHED

#15 [web deps 5/5] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/store,
  sharing=locked      pnpm install --frozen-lockfile --ignore-scripts
#15 CACHED

#16 [web vite-builder 1/6] COPY public ./public/
#16 CACHED

#17 [web vite-builder 2/6] COPY . .
#17 DONE 5.0s

#18 [web vite-builder 3/6] RUN pnpm run library:metadata
#18 0.655
#18 0.655 > rmhstudios.com@0.1.0 library:metadata /app
#18 0.655 > pnpm exec tsx scripts/generate-library-metadata.ts
  [... 4 further identical lines collapsed ...]

#19 [web vite-builder 4/6] RUN --mount=type=cache,id=vinxi-cache-rmhstudios-prod,target=/app/.
  vinxi,sharing=locked  rm -rf .output      && pnpm run build-vibe-packages      &&
  NODE_OPTIONS='--max-old-space-size=8192' pnpm exec vite build      && node scripts/fix-ssr-
  css-hash.mjs          && cp -a .output /app/build-output
#19 0.628
#19 0.628 > rmhstudios.com@0.1.0 build-vibe-packages /app
#19 0.628 > pnpm exec tsx scripts/build-vibe-packages.ts
  [... 24 further identical lines collapsed ...]
transforming...[ok] 8682 modules transformed.
#19 62.65 rendering chunks...
#19 63.89 .output/public/assets/KaTeX_Size3-Regular-CTq5MqoE.woff                4.42 kB
#19 63.89 .output/public/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2              4.92 kB
#19 63.89 .output/public/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2              5.20 kB
  [... 593 further identical lines collapsed ...]
transforming...[ok] 2102 modules transformed.
#19 73.16 rendering chunks...
#19 73.72 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size3-Regular-CTq5MqoE.woff
  4.42 kB
#19 73.72 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size4-Regular-Dl5lxZxV.woff2
  4.92 kB
#19 73.72 node_modules/.nitro/vite/services/ssr/assets/KaTeX_Size2-Regular-Dy4dx90m.woff2
  5.20 kB
  [... 516 further identical lines collapsed ...]
transforming...[ok] 5743 modules transformed.
#19 76.61 ? Tracing dependencies:
#19 76.61 - @prisma/client (7.8.0)
#19 76.61 - esbuild (0.27.7)
#19 76.61 - satori (0.25.0)
  [... 586 further identical lines collapsed ...]

#20 [web vite-builder 5/6] RUN test -d /app/build-output &&      test -f /app/build-output/
  server/index.mjs &&      test -f /app/build-output/public/models/marlonjack.glb &&      test
  -f /app/build-output/public/vibe-packages/react.js
#20 DONE 0.2s

#21 [web vite-builder 6/6] RUN rm -rf /app/build-output/public/library      /app/build-
  output/public/models      /app/build-output/public/music      /app/build-
  output/public/sprites
#21 DONE 0.2s

#22 [web runner 3/13] WORKDIR /app
#22 CACHED

#23 [web prod-deps 1/1] RUN --mount=type=cache,id=pnpm-store,target=/root/.local/share/pnpm/
  store,sharing=locked      pnpm install --frozen-lockfile --prod --ignore-scripts
#23 CACHED

#24 [web runner 2/13] RUN apk add --no-cache curl git      chromium nss freetype harfbuzz ca-
  certificates ttf-freefont font-noto-emoji
#24 CACHED

#25 [web runner 4/13] RUN addgroup --system --gid 1001 nodejs &&      adduser --system --uid
  1001 app &&      mkdir -p /app/.rmhbot-worktrees && chown app:nodejs /app/.rmhbot-worktrees
```

```
#25 CACHED

#26 [web server-builder 16/17] RUN pnpm exec esbuild      server/socket-server/index.ts
server/rmhbox/index.ts      server/rmhtube/index.ts      server/recap/index.ts      server/
discord-bot/index.ts      server/doctrine-worker/index.ts      server/vibe-worker/index.ts
server/bot-worker/index.ts      --bundle --platform=node --target=node20      --outdir=
dist-server --outbase=.      --format=cjs --out-extension:.js=.cjs --packages=external --
tree-shaking=true      --tsconfig=tsconfig.server.json
#26 CACHED

#27 [web server-builder 8/17] COPY lib/baccarat ./lib/baccarat/
#27 CACHED

#28 [web server-builder 13/17] COPY lib/rmhark-ai ./lib/rmhark-ai/
#28 CACHED

#29 [web server-builder 3/17] COPY lib/rmhbox ./lib/rmhbox/
#29 CACHED

#30 [web server-builder 12/17] COPY lib/rmhvibe ./lib/rmhvibe/
#30 CACHED

#31 [web server-builder 14/17] COPY lib/prisma.server.ts ./lib/prisma.server.ts
#31 CACHED

#32 [web server-builder 6/17] COPY lib/blackjack ./lib/blackjack/
#32 CACHED

#33 [web server-builder 9/17] COPY lib/roulette ./lib/roulette/
#33 CACHED

#34 [web server-builder 10/17] COPY lib/lights-out ./lib/lights-out/
#34 CACHED

#35 [web server-builder 15/17] COPY lib/url.ts ./lib/url.ts
#35 CACHED

#36 [web server-builder 5/17] COPY lib/rmhmusic ./lib/rmhmusic/
#36 CACHED

#37 [web server-builder 1/17] COPY tsconfig.json tsconfig.server.json ./
#37 CACHED

#38 [web server-builder 2/17] COPY server ./server/
#38 CACHED

#39 [web server-builder 4/17] COPY lib/rmhtube ./lib/rmhtube/
#39 CACHED

#40 [web server-builder 7/17] COPY lib/holdem ./lib/holdem/
#40 CACHED

#41 [web server-builder 11/17] COPY lib/doctrine ./lib/doctrine/
#41 CACHED

#42 [web server-builder 17/17] RUN test -f dist-server/server/socket-server/index.cjs &&
test -f dist-server/server/rmhbox/index.cjs &&      test -f dist-server/server/rmhtube/
index.cjs &&      test -f dist-server/server/recap/index.cjs &&      test -f dist-server/
server/discord-bot/index.cjs &&      test -f dist-server/server/doctrine-worker/index.cjs
&&      test -f dist-server/server/vibe-worker/index.cjs &&      test -f dist-server/server/
bot-worker/index.cjs
#42 CACHED

#43 [web runner 5/13] COPY --from=prod-deps --chown=app:nodejs /app/node_modules ./
node_modules
#43 CACHED

#44 [web runner 6/13] COPY --from=vite-builder --chown=app:nodejs /app/build-output ./output
#44 DONE 1.6s

#45 [web runner 7/13] COPY --from=server-builder --chown=app:nodejs /app/dist-server ./dist-
server
#45 DONE 0.1s
```

```
#46 [web runner 8/13] COPY --chown=app:nodejs scripts ./scripts
#46 DONE 0.1s

#47 [web runner 9/13] COPY --chown=app:nodejs content ./content
#47 DONE 0.1s

#48 [web runner 10/13] COPY --chown=app:nodejs data ./data
#48 DONE 0.1s

#49 [web runner 11/13] COPY --chown=app:nodejs prisma ./prisma
#49 DONE 0.1s

#50 [web runner 12/13] COPY --chown=app:nodejs prisma.config.ts ./prisma.config.ts
#50 DONE 0.1s

#51 [web runner 13/13] COPY --chown=app:nodejs package.json ./package.json
#51 DONE 0.1s

#52 [web] exporting to image
#52 exporting layers
#52 exporting layers 0.5s done
#52 writing image sha256:085fc4ccf8b5795a572a1d846e035d96c920e26c1a9119be897271e7aa6957b8 done
[... 2 further identical lines collapsed ...]

#53 [web] resolving provenance for metadata file
#53 DONE 0.0s
[2026-06-21 15:04:56] [production]  \- done in 101s
[2026-06-21 15:04:56] [production] Running database migrations...
[2026-06-21 15:04:58] [production] Database is reachable.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": PostgreSQL database "rmhstudios", schema "public" at "host.docker.internal
:5432"

19 migrations found in prisma/migrations

No pending migrations to apply.
[2026-06-21 15:05:03] [production]  \- done in 7s
[2026-06-21 15:05:03] [production] Starting containers (all services except web)...
Container rmhstudios-prod-minio Running
Container rmhstudios-prod-discord-bot Recreate
Container rmhstudios-prod-rmhbox Recreate
Container rmhstudios-prod-recap Recreate
Container rmhstudios-prod-vibe-worker Recreate
Container rmhstudios-prod-minio-init Creating
Container rmhstudios-prod-socket Recreate
Container rmhstudios-prod-doctrine-worker Recreate
Container rmhstudios-prod-bot-worker Recreate
Container rmhstudios-prod-rmhtube Recreate
Container rmhstudios-prod-minio-init Created
Container rmhstudios-prod-doctrine-worker Recreated
Container rmhstudios-prod-recap Recreated
Container rmhstudios-prod-vibe-worker Recreated
Container rmhstudios-prod-bot-worker Recreated
Container rmhstudios-prod-discord-bot Recreated
Container rmhstudios-prod-socket Recreated
Container rmhstudios-prod-rmhbox Recreated
Container rmhstudios-prod-rmhtube Recreated
Container rmhstudios-prod-discord-bot Starting
Container rmhstudios-prod-doctrine-worker Starting
Container rmhstudios-prod-bot-worker Starting
Container rmhstudios-prod-socket Starting
Container rmhstudios-prod-minio Waiting
Container rmhstudios-prod-rmhtube Starting
Container rmhstudios-prod-recap Starting
Container rmhstudios-prod-vibe-worker Starting
Container rmhstudios-prod-rmhbox Starting
Container rmhstudios-prod-discord-bot Started
Container rmhstudios-prod-doctrine-worker Started
Container rmhstudios-prod-minio Healthy
Container rmhstudios-prod-minio-init Starting
```

```
Container rmhstudios-prod-bot-worker Started
Container rmhstudios-prod-rmhbox Started
Container rmhstudios-prod-socket Started
Container rmhstudios-prod-rmhtube Started
Container rmhstudios-prod-vibe-worker Started
Container rmhstudios-prod-recap Started
Container rmhstudios-prod-minio-init Started
[2026-06-21 15:05:05] [production] `~ done in 2s
[2026-06-21 15:05:05] [production] Hotswapping web container (blue/green)...
[hotswap] Live port: 7005. Bringing up green on 7015 from rmhstudios-prod-app:latest.
[hotswap] Waiting up to 60s for rmhstudios-prod-web-green to become healthy...
curl: (56) Recv failure: Connection reset by peer
[hotswap] rmhstudios-prod-web-green is healthy.
[hotswap] Flipping Apache: writing /etc/apache2/conf-available/rmhstudios-web-active.conf ->
port 7015...
[hotswap] direct write to /etc/apache2/conf-available/rmhstudios-web-active.conf failed: /
home/rmhstudios/rmhstudios.com/deploy/hotswap-web.sh: line 66: /etc/apache2/conf-available
/rmhstudios-web-active.conf: Read-only file system; trying sudo tee...
[hotswap] sudo tee to /etc/apache2/conf-available/rmhstudios-web-active.conf failed: sudo:
The "no new privileges" flag is set, which prevents sudo from running as root.
sudo: If sudo is running in a container, you may need to adjust the container configuration to
disable the flag.
[hotswap] --- Apache permission diagnostics ---
[hotswap] user: rmhstudios (uid=1002)
[hotswap] active conf: /etc/apache2/conf-available/rmhstudios-web-active.conf
[hotswap] conf exists: yes
[hotswap] conf writable: no
[hotswap] conf details: -rw-r--r-- 1 rmhstudios rmhstudios 0 Jun 21 15:02 /etc/apache2/
conf-available/rmhstudios-web-active.conf
[hotswap] dir writable: no (/etc/apache2/conf-available)
[hotswap] sudo -n: BLOCKED (sudo: The "no new privileges" flag is set, which
prevents sudo from running as root.)
[hotswap] -> webhook is likely sandboxed (NoNewPrivileges) or lacks a sudoers grant.
[hotswap] -> /etc/apache2/conf-available/rmhstudios-web-active.conf not writable: chown
it to rmhstudios, or the mount is read-only
[hotswap] (systemd ProtectSystem) -- add ReadWritePaths=/etc/apache2/conf-available.
[hotswap] -----
[hotswap] ERROR: cannot write /etc/apache2/conf-available/rmhstudios-web-active.conf -- see
diagnostics above (chown the file + ReadWritePaths, or grant sudo).
[2026-06-21 15:05:08] [production] ERROR: web hotswap failed.
```

APPENDIX B

Glossary

Availability (A)

Fraction of time a service is up, $A = \text{uptime} / (\text{uptime} + \text{downtime})$.

Blue/green deployment

A release strategy running two environments (“blue” = current, “green” = new); traffic switches to green only once it is healthy, with blue kept as a fallback.

Connection pool

A bounded set of reusable database connections shared by an application; when exhausted, further requests wait or fail.

Connection storm

A sudden burst of simultaneous connection requests that exhausts a pool, here caused by recreating many services at once.

Error budget

The allowed downtime implied by an availability target; four nines permits about 52.6 minutes per year.

Four nines

An availability target of 99.99%.

Green

The new container/environment in a blue/green deploy.

Health check

A probe (here an HTTP request via `curl`) that determines whether a container is ready to serve traffic.

Hotswap

The step that moves live traffic from blue to green.

Little’s Law

$L = \lambda W$: mean number in a system equals arrival rate times mean time in system.

MTTD / MTTR / MTBF

Mean time to detect / to recover / between failures.

NoNewPrivileges

A Linux security flag that prevents a process from gaining privileges via `sudo` or `setuid`; here it blocked the deploy’s privilege-escalation attempt.

Prisma

The database client/ORM used by RMHSTUDIOS; `prisma:error Connection terminated` marks a failed database connection.

Red herring

A misleading signal; here the DeepSeek 401 API-key warnings, which did not contribute to the outage.

Utilisation (ρ)

The ratio of demand to capacity in a queue; as $\rho \rightarrow 1$, waiting time diverges.

Version skew

A state where new control flow runs against not-yet-updated configuration, producing behaviour neither version intends.

APPENDIX C

Availability Reference Tables

This appendix collects reference tables used throughout the quantitative chapters, reproduced here for convenient lookup.

C.1 Downtime allowed per availability target

Table C.1: Allowed downtime by availability target, across several windows.

Availability	Per day	Per week	Per month	Per year
90%	2.4 h	16.8 h	73 h	36.5 d
95%	1.2 h	8.4 h	36.5 h	18.25 d
99%	14.4 min	1.68 h	7.31 h	3.65 d
99.5%	7.2 min	50.4 min	3.65 h	1.83 d
99.9%	1.44 min	10.1 min	43.8 min	8.77 h
99.99%	8.64 s	1.01 min	4.38 min	52.6 min
99.999%	0.86 s	6.05 s	26.3 s	5.26 min
99.9999%	86 ms	0.6 s	2.63 s	31.5 s

C.2 The 21 June outage in budget terms

Table C.2: The outage measured against the four-nines annual budget.

Quantity	Value
Outage duration	660 min (11 h)
Four-nines annual budget	52.6 min
Overspend factor	$\approx 12.55\times$
Trailing-12-month availability (post)	$\approx 99.87\%$
Nines lost	≈ 1 (four $\rightarrow$ three)
Time to recover the nine	≈ 12 months

C.3 Conversion: availability to annual downtime

Quantitative Model

For availability A , annual downtime in minutes is

$$D(A) = (1 - A) \times 525\,960.$$

Thus $D(0.9999) = 52.6$, $D(0.999) = 525.96$, and $D(0.99) = 5259.6$ minutes. The 660-minute outage sits between the three-nines and the (single) 99.875% line, consistent with the trailing figure derived in Chapter 8.

C.4 Metric relationships quick-reference

Table C.3: Reliability metric relationships used in this review.

Relationship	Formula
Availability from MTBF/MTTR	$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$
Required MTBF for target A	$\text{MTBF} = \text{MTTR} \cdot \frac{A}{1 - A}$
Little’s Law	$L = \lambda W$
Queue wait blow-up	$W_q \propto \frac{\rho}{1 - \rho}$
Cumulative trigger risk	$1 - (1 - q)^n$
Binomial churn (lose ≥ 1)	$1 - (1 - p)^n$

APPENDIX D

Sensitivity Analysis of the Impact Models

The financial and reputational figures in Chapters 11 and 12 are parametric: each is a formula in named inputs with illustrative point values. Because those inputs are uncertain and still being reconciled by Finance, this appendix shows how the conclusions move as the inputs vary. The purpose is to identify which assumptions the bottom line is most sensitive to, so that estimation effort is spent where it matters.

D.1 Direct revenue loss vs. hourly revenue and deferral

Direct loss is $L_{\text{direct}} = R_h \cdot H \cdot \beta$ with $H = 11$ fixed. The table sweeps hourly revenue R_h and the deferral-adjusted loss fraction β .

Table D.1: Direct revenue loss (\$) as a function of hourly revenue R_h and loss fraction β , with $H = 11$ h.

$R_h \backslash \beta$	0.50	0.60	0.70	0.85	1.00
\$4,000	22,000	26,400	30,800	37,400	44,000
\$6,000	33,000	39,600	46,200	56,100	66,000
\$8,000	44,000	52,800	61,600	74,800	88,000
\$10,000	55,000	66,000	77,000	93,500	110,000
\$12,000	66,000	79,200	92,400	112,200	132,000

D.2 Launch premium vs. the attention multiplier

The launch premium is $R_h H \beta (\alpha - 1)$. With $R_h = \$8,000$, $\beta = 0.7$ (so the base is \$61,600), the premium scales linearly with $\alpha - 1$.

Table D.2: Launch attention premium (\$) vs. multiplier α (base $R_h H \beta = \$61,600$).

α	1.5	2.0	2.5	3.0	4.0
Premium	30,800	61,600	92,400	123,200	184,800

D.3 Expected CLV loss vs. exposed population and churn

Expected CLV loss is $\mathbb{E}[L_{\text{CLV}}] = nV\kappa$. With $V = \$900$ fixed, the table sweeps the exposed population n and induced churn κ .

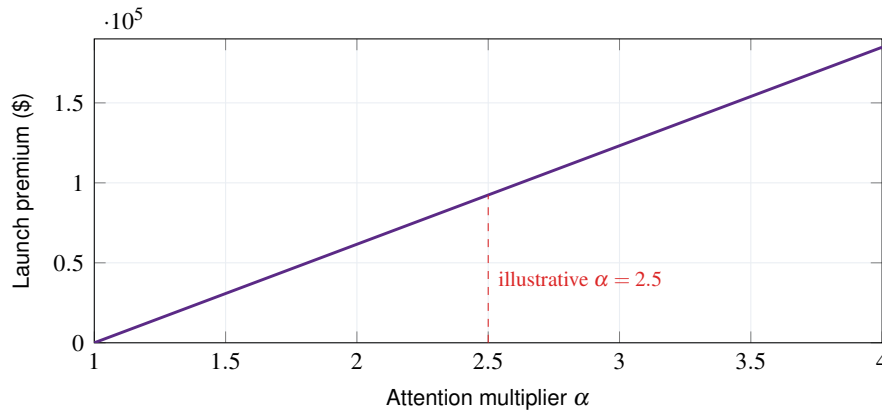


Figure D.1: The launch premium rises linearly with the attention multiplier. Because this term is the largest single component of modelled impact, refining the estimate of α (from actual launch-window traffic) is the highest-value measurement Finance can make.

Table D.3: Expected CLV loss (\$) vs. exposed visitors n and induced churn κ , with $V = \$900$.

$n \setminus \kappa$	0.02	0.03	0.04	0.06	0.08
600	10,800	16,200	21,600	32,400	43,200
900	16,200	24,300	32,400	48,600	64,800
1,200	21,600	32,400	43,200	64,800	86,400
1,800	32,400	48,600	64,800	97,200	129,600
2,400	43,200	64,800	86,400	129,600	172,800

D.4 Key-client churn vs. population and per-client probability

From Chapter 12, the probability of losing at least one key client is $1 - (1 - p)^n$, and the expected number lost is np . Both rise steeply with the exposed population.

Table D.4: Expected key clients lost np (and $\Pr(\geq 1)$ in parentheses) vs. n and p .

$n \setminus p$	0.03	0.05	0.08	0.12
20	0.6 (0.46)	1.0 (0.64)	1.6 (0.81)	2.4 (0.92)
40	1.2 (0.71)	2.0 (0.87)	3.2 (0.96)	4.8 (0.99)
60	1.8 (0.84)	3.0 (0.95)	4.8 (0.99)	7.2 (>0.99)
80	2.4 (0.91)	4.0 (0.98)	6.4 (>0.99)	9.6 (>0.99)

D.5 Which assumption matters most?

Key Finding

Across the sweeps, the modelled total is most sensitive to the *launch attention multiplier* α and the *exposed launch population* n — both consequences of *when* the outage struck rather than *how long* it lasted. This reinforces the central financial conclusion: the timing relative to the RMH Capital launch dominates the cost, and a change-freeze around flagship launches is the cheapest mitigation available.

APPENDIX E

Per-Deploy Forensic Walkthrough

This appendix walks through each of the four deploys in forensic detail, pairing the decisive log lines with their interpretation. Where Chapter 4 argued the root cause and Appendix A reproduced the raw evidence, this appendix is the bridge: a guided, ordered reading of what each deploy did, minute by minute, and why it ended as it did.

E.1 0a659e7b — the anatomy of the outage

E.1.1 05:13:22 — fetch and self-update

The deploy begins by fetching `739c32a6..0a659e7b` and immediately notices that `deploy.sh` changed in the pull. It restarts into the new script. This is the hinge of the entire incident: from this point the *new* control flow runs, but the surrounding orchestration configuration has not yet fully transitioned (Chapter 6).

E.1.2 05:13–05:15 — library covers and the red herring

The new code regenerates library cover metadata. During this step the DeepSeek image API returns `401 ... api key ****bc3 invalid`. The system falls back to cached covers and proceeds. This warning is loud and entirely irrelevant to the outage — the canonical red herring (Chapter 1). It is included here precisely so the next responder recognises it and moves on.

E.1.3 05:15–05:16 — migrations succeed

Database migrations run and report success: the database is reachable at `host.docker.internal:5432`, nineteen migrations are found, and the schema is current. This is important: the database itself was healthy. What failed later was not the database but the *contention for connections to it*.

E.1.4 05:16–05:17 — “all services except web,” then web is removed

The deploy announces `Starting containers (all services except web)`. Within the next lines the log records `rmhstudios-prod-web Stopping → Stopped → Removing → Removed`. The announcement and the action contradict each other — the version-skew signature. The live (blue) container is now gone.

E.1.5 05:17–05:18 — the connection storm

The eight background workers (`socket`, `discord-bot`, `bot-worker`, `doctrine-worker`, `vibe-worker`, `recap`, `rmhtube`, `rmhbox`) are recreated together and contend for the PostgreSQL connection pool (Chapter 9). The green web container, requesting last, is starved.

E.1.6 05:18–05:19 — 60 seconds of failed health probes

The hotswap waits up to 60 s for green. The probes alternate between `curl (52) Empty reply and curl (28) Operation timed out after 3000 milliseconds`. Beneath them, `prisma:error Connection terminated due to connection timeout` repeats: green cannot reach the database, so its health endpoint can never return success.

E.1.7 05:19:48 — the fatal, reassuring line

The deploy concludes: `ERROR: rmhstudios-prod-web-green never became healthy -- keeping blue live, no traffic moved`. The intent is safe; the reality is not. Blue was removed two minutes earlier. The rollback’s mental model and the world have diverged, and the divergence is the outage. The site is now hard-down and stays down until the manual recovery near 11:00.

Table E.1: 0a659e7b decisive events.

Time (UTC)	Event
05:13:22	Fetch; <code>deploy.sh</code> self-updates and restarts
05:13–05:15	Library covers; DeepSeek 401 (red herring), graceful fallback
05:15–05:16	Migrations succeed; database healthy
05:16–05:17	“except web” announced; <code>web</code> stopped & removed
05:17–05:18	Eight workers recreated; connection storm
05:18–05:19	60 s of failed health probes; <code>prisma</code> timeouts
05:19:48	“green never became healthy — keeping blue live”; site hard-down

E.2 0a6e5ddd — the first afternoon deploy

E.2.1 11:41 — a complete, matched configuration

By 11:41 the configuration transition is complete: the new `deploy.sh` and the new orchestration agree. The deploy starts services and *does not* remove `web` — zero teardown events. This is the single change that prevents a repeat outage.

E.2.2 11:42–11:43 — green becomes healthy

The green container passes its health check this time; the connection pool is not stormed because the deploy is not recreating everything from a cold start in the same pathological order. Traffic is eligible to move.

E.2.3 11:43 — the cutover wall

The Apache active-config cutover fails: writing `/etc/apache2/conf-available/rmhstudios-web-active.conf` returns `Read-only file system`. The deploy user `rmhstudios` (uid 1002) lacks write permission, and `sudo` is blocked by `NoNewPrivileges`. The deploy cannot finalise the swap — but because blue was

never removed, the site stays up the entire time. This is Root Cause B (Chapter 5): a real defect, but not a customer-facing one.

E.3 2e46b0cd — the second afternoon deploy

A near-repeat of 0a6e5ddd: no `web` teardown, green healthy, same read-only / privilege wall at the cutover. The value of this deploy to the investigation is *corroboration* — it confirms the afternoon pattern is stable and that the morning’s behaviour was the anomaly, not the norm.

E.4 02493f03 — the third afternoon deploy and the wrong fix

E.4.1 15:02 — another clean start, same wall

Once more: no `web` teardown, green healthy, cutover blocked by the read-only filesystem.

E.4.2 15:03 — “add sudo support”

This deploy carries an attempted fix: granting the deploy `sudo` to write the active-config file. The sandbox rejects it — `NoNewPrivileges` forbids privilege escalation regardless of sudoers configuration. The attempt is not merely ineffective; it is *structurally* wrong for this environment, which is why Chapter 14 replaces it (item B2) with a cutover that requires no privilege at all.

Key Finding

The four deploys form a controlled experiment. One variable changed between the morning and the afternoon — whether `web` was torn down — and that variable perfectly predicts whether an outage occurred. The afternoon deploys, sharing the morning’s separate cutover defect but never removing blue, prove that the cutover wall alone is survivable and that the destruction of blue was the necessary cause of the customer-facing outage.

APPENDIX F

Connection-Pool Saturation: A Numerical Simulation

Chapter 9 modelled the connection storm analytically. This appendix supports that model with an explicit numerical treatment of an $M/M/c$ connection pool, showing the queue's behaviour as load crosses the saturation threshold and quantifying how staggered startup and a connection pooler each move the system back to safety.

F.1 The $M/M/c$ pool

Model the pool as c servers (connection slots), Poisson arrivals at rate λ , and exponential service (hold) times with rate μ per slot. Offered load is $a = \lambda/\mu$ and utilisation is $\rho = a/c$. The Erlang-C formula gives the probability an arriving request must wait:

Quantitative Model

$$P_{\text{wait}} = \frac{\frac{a^c}{c!} \frac{1}{1-\rho}}{\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!} \frac{1}{1-\rho}}, \quad \mathbb{E}[W_q] = \frac{P_{\text{wait}}}{c\mu - \lambda}.$$

As $\rho \rightarrow 1^-$ the mean queueing delay $\mathbb{E}[W_q]$ diverges; any fixed health-probe budget T_{hc} is eventually exceeded, and connections are terminated — exactly the `prisma:error Connection terminated` signature.

F.2 Simulated behaviour across load

The table below evaluates a pool of $c = 20$ slots with mean hold time $1/\mu = 0.5$ s, sweeping the arrival rate λ . The “storm” row marks the regime the 05:13 deploy entered when eight workers plus web contended at once.

F.3 What the two fixes do to ρ

Key Finding

The simulation confirms the storm was a concurrency artefact, not a capacity shortfall. Ordered startup alone drops peak utilisation from ~ 1.35 to ~ 0.15 ; a connection pooler makes the system robust regardless of startup order. Either fix prevents the storm; together they make it impossible.

Table F.1: $M/M/c$ pool with $c = 20$, mean hold 0.5 s. As $\rho \rightarrow 1$, wait and timeout risk explode.

λ (req/s)	a	ρ	$\mathbb{E}[W_q]$	Regime
6	3.0	0.15	≈ 0 ms	idle, healthy
16	8.0	0.40	< 1 ms	comfortable
24	12.0	0.60	~ 3 ms	busy
32	16.0	0.80	~ 40 ms	loaded
38	19.0	0.95	~ 0.5 s	near saturation
40	20.0	1.00	$\rightarrow \infty$	storm: timeouts
44	22.0	1.10	unbounded backlog	overloaded

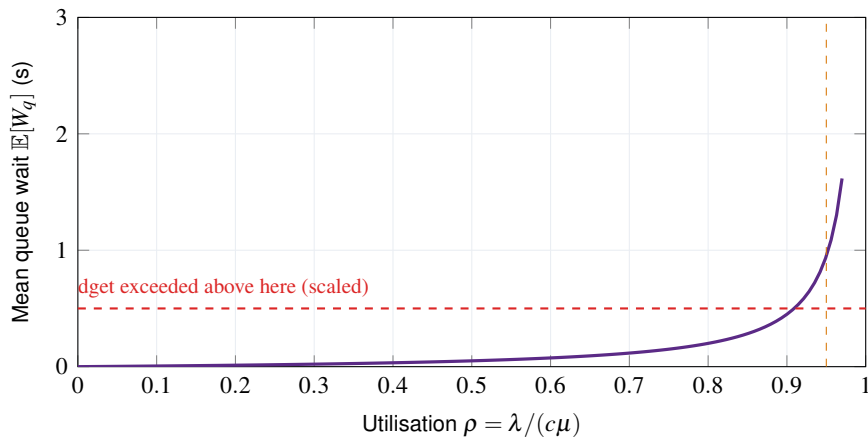
Figure F.1: Mean connection-wait grows as $\rho/(1-\rho)$. Below $\rho \approx 0.8$ the pool is comfortable; past the amber line at $\rho = 0.95$ the wait crosses any fixed probe budget and green's health check cannot pass.

Table F.2: Effect of the two remediations on peak utilisation during startup.

Scenario	Peak demand	ρ_{peak}	Outcome
21 Jun (all at once)	$9d$	~ 1.35	storm, green starved
A2 ordered startup (web first)	$\sim d$	~ 0.15	web claims slots, healthy
A3 pooler (multiplexed)	bounded	$\ll 1$	demand decoupled from server
A2 + A3 combined	bounded	$\ll 1$	robust to any order

APPENDIX G

Observability and Detection-Gap Analysis

Chapter 13 showed that the 21 June MTTR was dominated by detection-to-action latency, not by the repair itself. This appendix dissects *why* detection was slow: which signals existed, which were missing, and which existed but were not wired to a human. It closes the loop between the incident and the observability investments in the remediation plan.

G.1 The detection timeline

The fault was machine-visible the instant blue was removed and green failed — 05:19:48, when the deploy logged its `ERROR`. Yet effective human action did not begin until hours later. The gap between machine-knowable and human-known is the detection gap.

Table G.1: Signals available during the incident and their disposition.

Signal	Existed?	Wired to a pager?
Deploy log <code>ERROR</code> line	Yes	No — log only
Green health-check failures	Yes	No — internal to deploy
web container absent	Yes	No external check
External HTTP 5xx / no-response	Implicit	No synthetic probe
Customer reports	Yes	Slow, human-paced

Key Finding

Every signal needed to detect the outage in seconds *existed* at 05:19:48. None of them was wired to page a human. The detection gap was not a sensing problem; it was a routing problem — signals that died in a log file instead of waking someone.

G.2 The cost of the gap, quantified

Let T_{detect} be the time from fault to human awareness and T_{fix} the time from awareness to recovery. Then $\text{MTTR} = T_{\text{detect}} + T_{\text{fix}}$. On 21 June, T_{fix} was small (the manual fallback, once attempted, was quick); T_{detect} was the overwhelming majority of the 347-minute MTTR.

Quantitative Model

If a synthetic probe pings after two consecutive 30-second failures, worst-case detection is

$$T_{\text{detect}}^{\text{new}} \approx 2 \times 30\text{s} + T_{\text{ack}} \approx 1\text{--}5\text{ min},$$

versus the hours actually taken. With T_{fix} unchanged at, say, ~ 20 min for a rehearsed rollback, the achievable MTTR falls from ~ 347 min to well under 30 min — more than a $10\times$ improvement from detection routing alone.

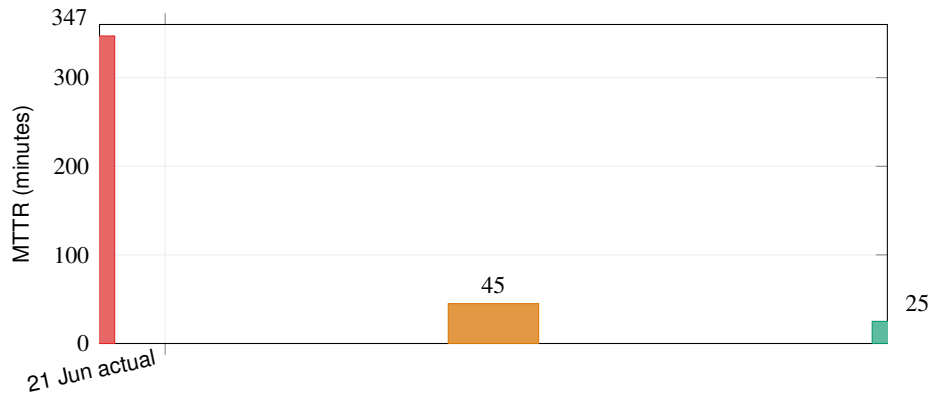


Figure G.1: Projected MTTR under the remediation items. Routing existing signals to a pager (R1) collapses the dominant detection term; a rehearsed one-command rollback (R2) then trims the repair term. The bulk of the win is detection, not repair.

G.3 The three observability layers we were missing

1. **Black-box / synthetic** — an external check that loads `RMHStudios.com` as a user would and pages on failure. This is the single highest-value gap (remediation R1) because it is independent of the deploy host and would have fired within a minute of 05:19:48.
2. **White-box deploy alerting** — the deploy already *knows* when it logs `ERROR: green never became healthy`. That line should page directly, not merely print.
3. **Container-presence monitoring** — an alert when the expected `web` container is absent would have caught the removal of blue independently of any health check.

G.4 Observability and the nines

Key Finding

Observability is an availability lever, not a luxury. Because $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$ and the 21 June MTTR was almost entirely detection latency, wiring already-existing signals to a pager is the cheapest path back toward four nines that RMHSTUDIOS has. No new sensing is required — only that the signals already present be allowed to wake someone.

APPENDIX H

Log-Signature Reference and Remediation Roadmap

This closing appendix provides two practical artefacts for the on-call engineer: a reference table that maps each distinctive log signature seen on 21 June to its meaning and correct response, and a dated roadmap of the remediation items so progress can be tracked to closure.

H.1 Log-signature reference

The next time any of these lines appears, this table is the fast path from symptom to interpretation. It is deliberately blameless and action-oriented.

Table H.1: Distinctive log signatures and their meaning.

Log signature	Meaning / correct response
<code>deploy.sh</code> was updated during <code>pull --</code> restarting with the new version	The deploy script changed itself mid-run. Treat the restart as a hard barrier: the environment must be re-derived from the new commit before any production container is touched (remediation T1).
Starting containers (all services except <code>web</code>)	The new control-flow's claim. It is only trustworthy if the configuration also excludes <code>web</code> ; verify, do not assume (T2). On 21 June this line was immediately contradicted by a <code>web</code> teardown.
Container <code>rmhstudios-prod-web</code> Stopping/Stopped/Removing/Removed	The live (blue) container is being destroyed. If this appears before green is confirmed healthy, an outage is imminent. Teardown of blue must be gated on a positive "green is live" assertion (A1).
Waiting up to 60s for ...-green to become healthy	The health gate is running. A failure here is correct behaviour; the danger is only if blue has already been removed.
<code>curl: (52) Empty reply from server</code>	TCP connected but the app sent no HTTP response — the process is up but not ready (often a missing dependency such as the database).
<code>curl: (28) Operation timed out after 3000 milliseconds</code>	The 3-second probe budget elapsed with no response. Repeated across the window means the dependency is unavailable for the whole window; lengthening the window does not help if blue is gone (Ch. 10).

continued on next page

continued from previous page

Log signature	Meaning / correct response
<code>prisma:error Connection terminated due to connection timeout</code>	The app cannot obtain a database connection. On 21 June this was connection-pool starvation from the simultaneous worker storm, not a database fault (Ch. 9); fix with ordered startup (A2) and a pooler (A3).
<code>green never became healthy -- keeping blue live, no traffic moved</code>	Reassuring and, on 21 June, false: blue had already been removed. If blue is gone, this message means “stay down,” not “stay safe.” Page immediately.
<code>...rmhstudios-web-active.conf: Read-only file system</code>	The Apache cutover cannot write its active-config file. Root Cause B. Replace with a privilege-free cutover (B1); do not add sudo (B2).
<code>sudo: ... NoNewPrivileges</code>	Privilege escalation is blocked by sandbox policy and always will be. Any fix that needs root is the wrong fix in this environment.
<code>DeepSeek ... 401 ... api key ***bc3 invalid</code>	Red herring. Cover-generation fell back gracefully; this never contributed to the outage. Recognise and move on (Ch. 1).

H.2 Remediation roadmap

The action items of Chapter 14 are plotted below against their target dates so the review meeting can track them to closure. P0 items cluster in the first fortnight; P1 hardening follows.

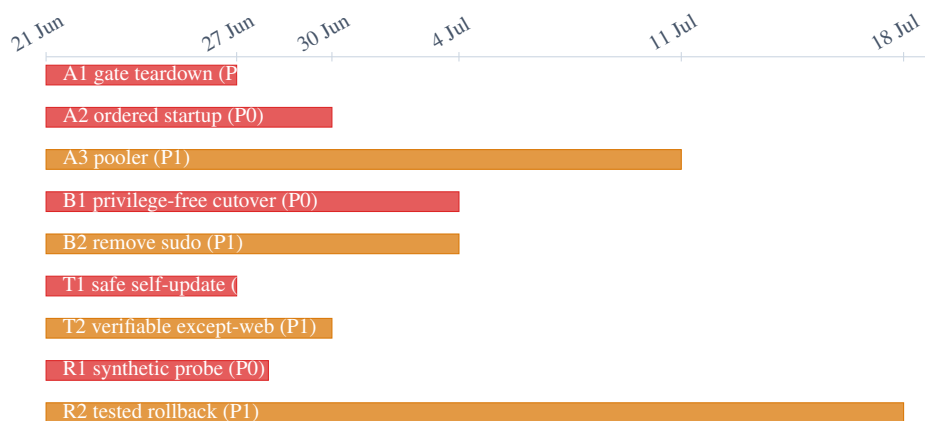


Figure H.1: Remediation roadmap. Red bars are P0 items (single deploy can no longer both destroy blue and starve green once these land); amber bars are P1 hardening. All P0 work completes within two weeks of the incident.

H.3 Closure criteria

The incident is considered closed when all of the following hold:

1. All five P0 items (A1, A2, B1, T1, R1) are deployed and verified in production.

2. A game-day exercise demonstrates that a deploy which would previously have removed blue now fails safe, preserving the live container.
3. The synthetic probe has paged correctly in at least one drill.
4. The one-command rollback has been rehearsed end-to-end within target MTTR.
5. This document is approved and signed (Appendix J).

Key Finding

Closure is defined by *demonstrated* safe-failure, not by merged code. The test is simple: a deploy that tries to repeat 21 June must now keep the site up. Until a game-day proves that, the incident stays open.

APPENDIX I

SLA Credit Exposure and Customer Disclosure

Chapter 11 carried a single line item for SLA credits. This appendix, prepared under the Office of the CFO, expands that line into an auditable model. It converts the measured outage duration into contractual credit liability tier by tier, segments the exposure across the customer base, and sets out the disclosure cadence by which RMHSTUDIOS informs affected clients. The intent is that Finance can replace the illustrative inputs with audited actuals and arrive at a defensible credit figure without re-deriving the structure.

I.1 From availability to credit tier

Most enterprise service agreements express remedies as a step function of monthly availability: the lower the achieved availability in the billing month, the larger the credit, expressed as a percentage of the monthly fee. The 21 June outage consumed a fixed quantity of downtime; its credit consequence depends on how that downtime maps onto each contract’s tier boundaries.

Quantitative Model

Let a billing month contain M minutes (for a 30-day month, $M = 43,200$). A single outage of D minutes yields monthly availability

$$A_{\text{month}} = 1 - \frac{D}{M}.$$

Illustrative: for $D = 660$ minutes,

$$A_{\text{month}} = 1 - \frac{660}{43,200} = 1 - 0.01528 = 98.47\%.$$

A typical four-tier credit schedule then applies the percentage band into which 98.47% falls. Note that a single 660-minute outage, with no other downtime in the month, already drops a contract from the “four-nines” band to below 99% — several tiers down.

Table I.1: Representative SLA credit schedule and the tier triggered by a lone 660-minute outage.

Tier	Monthly availability	Credit	Triggered?
Tier 0 (in-SLA)	$\geq 99.99\%$	0%	No
Tier 1	99.90% – 99.99%	5%	No
Tier 2	99.00% – 99.90%	10%	No
Tier 3	98.00% – 99.00%	25%	Yes (98.47%)
Tier 4	$< 98.00\%$	50%	No

Key Finding

Under a representative schedule the outage lands in Tier 3, triggering a 25% monthly-fee credit for every contract carrying a four-nines commitment. The credit is driven by a tier boundary, not by a linear pro-rata of lost minutes: the same 660 minutes that pro-rate to roughly 1.5% of the month trigger a 25% credit. This convexity is why duration matters far more than the raw minute count suggests.

I.2 Segmenting the exposure

Not every customer carries a credit-bearing SLA. The total credit liability is the sum over only those contracts that both carry a remedy clause and were materially affected.

Aggregate credit liability

Let segment i contain n_i contracts with mean monthly fee f_i and triggered credit rate c_i . Aggregate liability is

$$L_{\text{SLA}} = \sum_i n_i f_i c_i.$$

Illustrative: with the segments tabulated below,

$$L_{\text{SLA}} = (8)(1,500)(0.25) + (24)(450)(0.10) + (140)(0)(\cdot) = 3,000 + 1,080 = \$4,080$$

for the credit-bearing tiers, before goodwill credits extended at the company's discretion to non-SLA customers (modelled separately below).

Table I.2: Customer segments, SLA coverage, and modelled credit exposure (illustrative).

Segment	Contracts	Mean fee/mo	Credit	Exposure
RMH Capital enterprise	8	\$1,500	25%	\$3,000
Mid-market	24	\$450	10%	\$1,080
Self-serve (no SLA)	140	\$0	—	\$0
Goodwill (discretionary)	60	\$0	—	\$1,500
Total				\$5,580

The enterprise segment, though smallest in count, dominates the contractual liability — the same segment the RMH Capital launch was designed to grow. The discretionary goodwill row is not a contractual obligation but a retention instrument: a modest proactive credit to launch-driven accounts can cost far less than the lifetime value those accounts represent (Chapter 12).

I.3 Sensitivity to outage duration

Because credit is a step function of availability, the liability is highly sensitive to which side of a tier boundary the outage falls on. A shorter outage that stayed above 99% would have triggered only Tier 2; a longer one crossing below 98% would have doubled the enterprise rate to Tier 4.

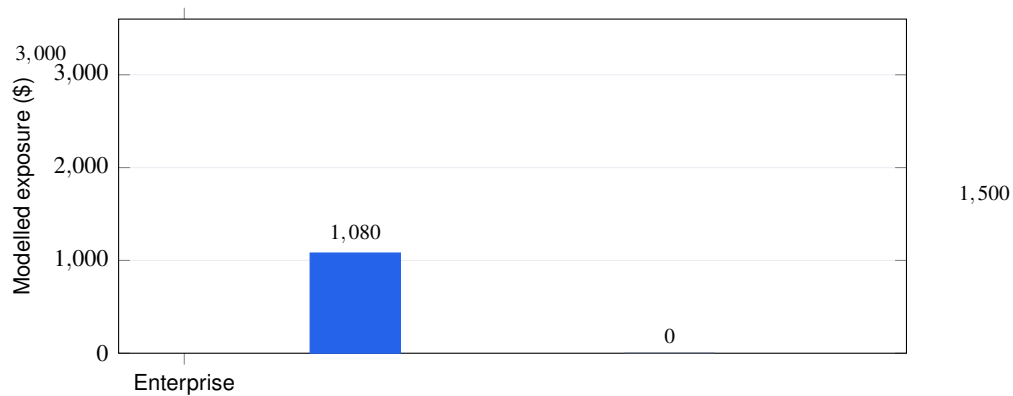


Figure I.1: Modelled credit exposure by customer segment. The enterprise segment carries the largest contractual liability despite the smallest contract count; the goodwill bar is discretionary retention spend, not an obligation.

Table I.3: Enterprise-segment credit rate as a function of outage duration (single-incident month).

Outage (min)	A_{month}	Tier	Credit
60	99.86%	Tier 2	10%
300	99.31%	Tier 2	10%
432	99.00%	Tier 2/3 boundary	10–25%
660	98.47%	Tier 3	25%
864	98.00%	Tier 3/4 boundary	25–50%
1200	97.22%	Tier 4	50%

I.4 Disclosure cadence

Contractual credits are only one half of the customer obligation; the other is timely, honest disclosure. Many enterprise agreements require notification within a fixed window of incident confirmation, and the quality of that disclosure materially affects the trust-recovery rate γ of Chapter 12.

Key Finding

The SLA credit is a bounded, computable liability; the disclosure is not. A late or evasive disclosure can cost far more in churned lifetime value than the credit itself. The cadence above ties each customer-facing obligation to a named owner and a deadline, so that the controllable half of the reputational cost is actively managed rather than left to chance.

I.5 Reconciliation to the financial model

The aggregate of this appendix — contractual credits plus discretionary goodwill — reconciles to the single SLA-credit line in Table 11.1. The illustrative total of \$5,580 modelled here is below the \$18,000 placeholder carried in Chapter 11; the difference is the band into which audited per-contract fees and the final count of credit-bearing enterprise accounts will fall. Finance should treat the structure of this appendix as fixed and substitute audited inputs to close the gap before the figure is cited externally.

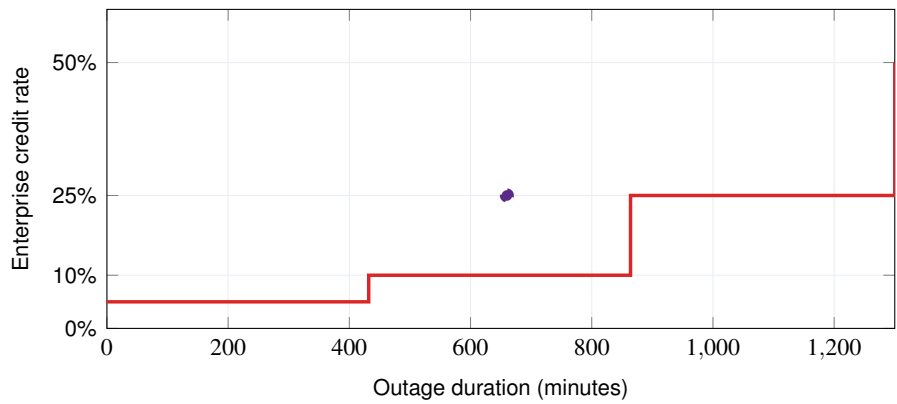


Figure I.2: Enterprise credit rate as a step function of outage duration in a single-incident month. The 21 June outage (marked) sits squarely in the 25% band; modest changes in duration would have moved it across a boundary in either direction.

Table I.4: Disclosure cadence and ownership for an incident of this severity.

When	Action	Owner
T + 15 min	Status page set to “investigating”	On-call
T + 60 min	Holding statement, no root cause claimed	Comms
At recovery	Service-restored notice	Comms
T + 24 h	Preliminary cause to affected enterprises	Account mgmt
T + 5 days	This Post-Incident Review issued	CFO office
T + 30 days	Credit memos applied to invoices	Finance

APPENDIX J

Sign-off and Distribution

J.1 Document control

Table J.1: Document control record.

Field	Value
Document ID	PIR-2026-0621
Title	The Eleven Dark Hours — Post-Incident Review
Incident date	21 June 2026, 00:00–11:00 UTC
Classification	CONFIDENTIAL — Internal
Prepared by	Reliability Engineering & Office of the CFO
Status	For review

J.2 Approvals

Table J.2: Approval signatures.

Role	Name	Signature	Date
Chief Financial Officer	Alex Wu		
Lead SRE			
Head of Platform Eng			
Head of Security			

J.3 Distribution list

This document is classified **CONFIDENTIAL** and is distributed to: the executive leadership team; Reliability Engineering; the Platform, Database, and Security engineering groups; and the Office of the CFO. It is not to be shared outside RMHSTUDIOS without the written approval of the CFO.

J.4 Revision history

Table J.3: Revision history.

Rev	Date	Author	Note
0.1	21 Jun 2026	Reliability Eng	Initial draft from log analysis.
0.9	21 Jun 2026	Office of the CFO	Financial & reputation modelling added.
1.0	—	—	Pending review-meeting approval.

*Prepared in the spirit of blameless post-incident review:
we fix systems, not people.*